MDMC
Master in Data Management and Curation

AREA SCIENCE PARK

SISSA

Introduction to Machine Learning · Day 1

Learning from Data

Trieste · Paolo Muratore · Monday 9 November 2026

CC BY

Overview

Learning from Data

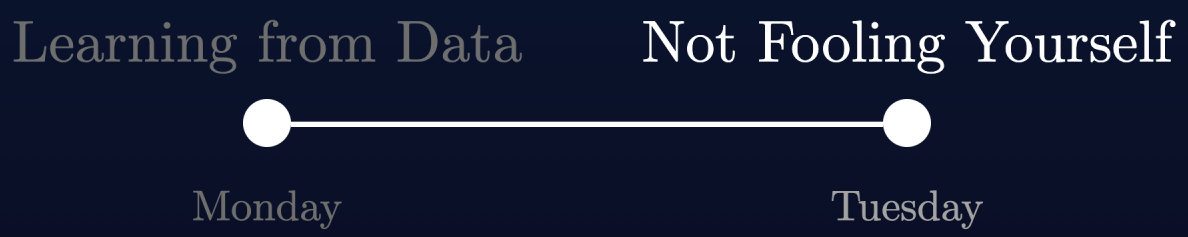


Monday

Where we turn learning into optimisation: models, losses and gradient descent



Overview



Where we learn to tell whether a model really works

Overview



Where we meet more models, look for structure without labels, and see how it is done for real

How We Will Work

Short lectures, a lot of guessing, and code every day

Guess first.

Before every reveal you commit to an answer. Wrong guesses are the useful ones.

Ask.

Interrupt me any time: your question is probably someone else's too.

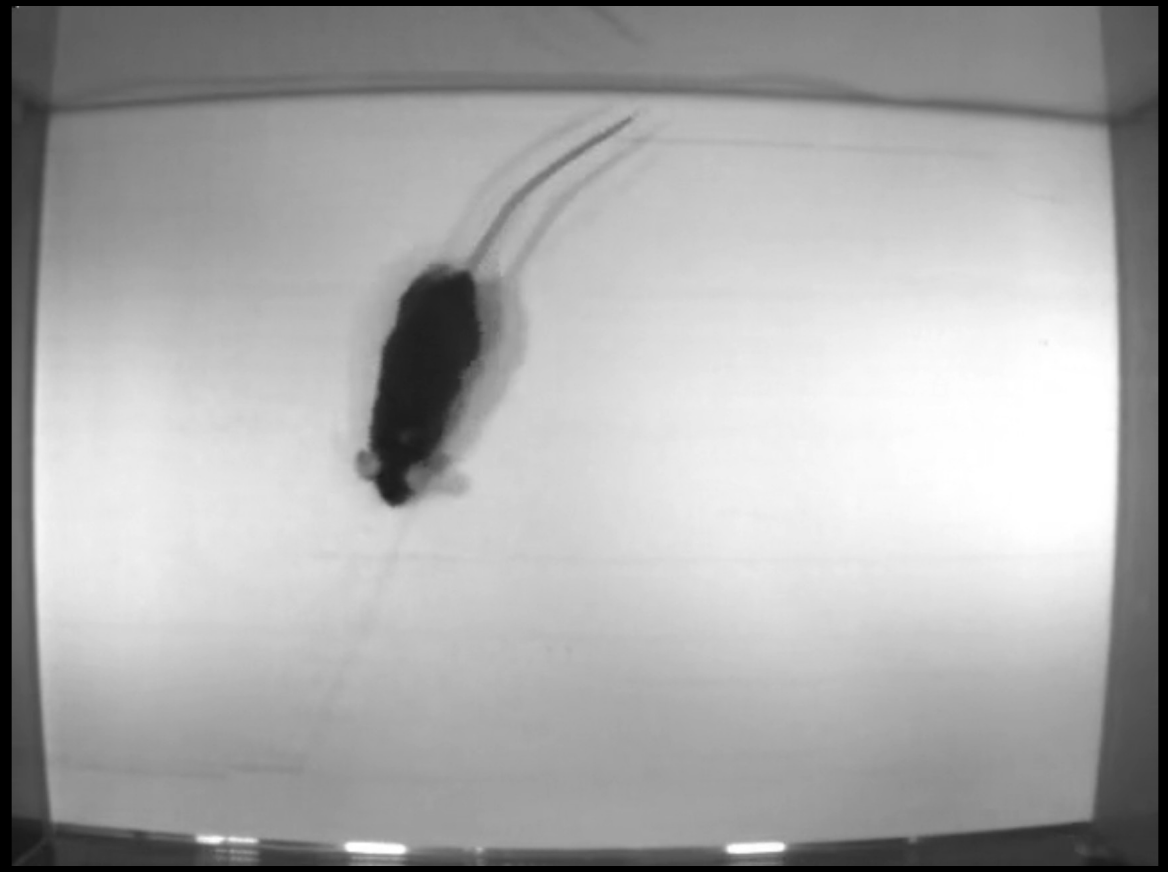
Build.

A Colab notebook every day, from core exercises to open challenges.

curator's corner: when the data, not the algorithm, decides what a model learns

Where Is the Snout?

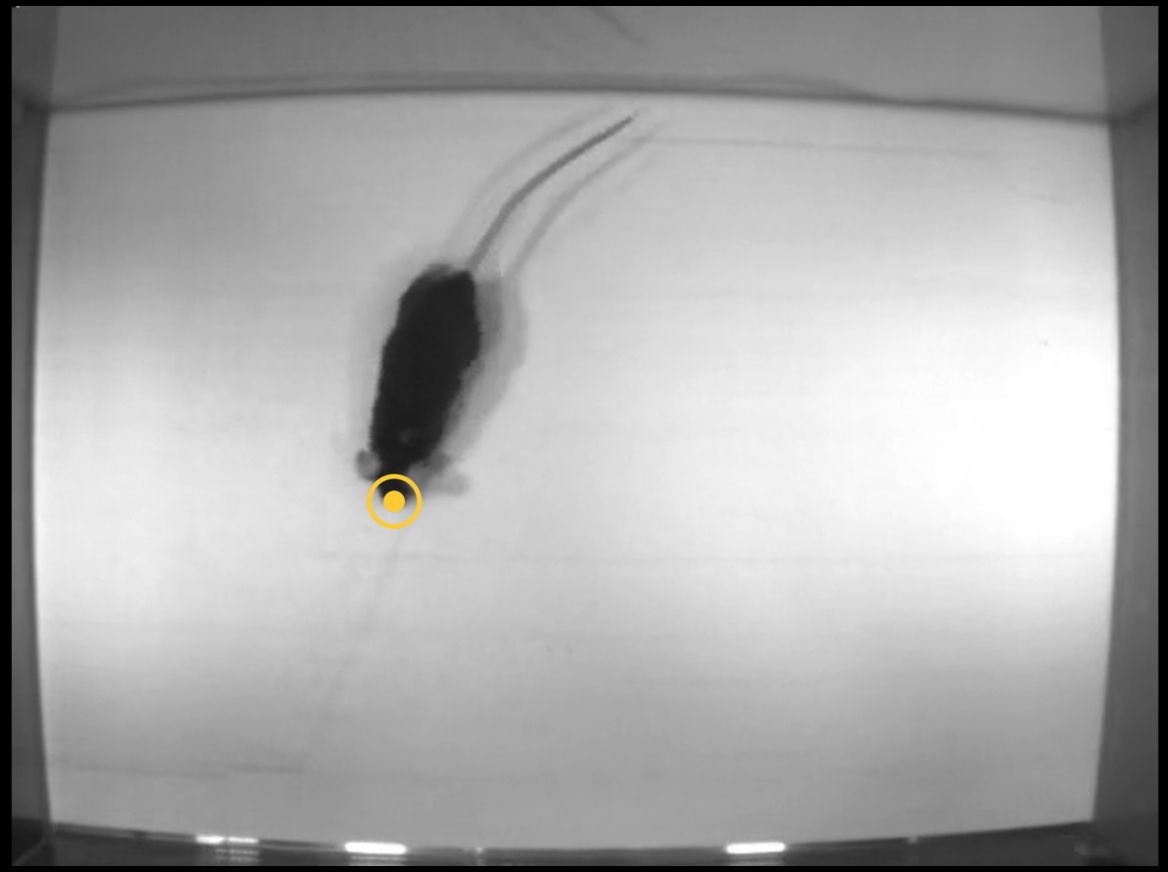
We want the snout's position in every frame: 108,000 frames per hour of video



Video: DeepLabCut open-field example, via Ye *et al.* (2024), CC BY 4.0

Where Is the Snout?

We want the snout's position in every frame: 108,000 frames per hour of video



Checking an answer is *easy*.

Writing the procedure that finds it is *hard*.

How would you write it?

Video: DeepLabCut open-field example, via Ye *et al.* (2024), CC BY 4.0

Rules We Might Try

Each rule works, until the next video

1. Find the dark blob,
take its pointy end

...until the tail is the pointier end

2. Follow it from the previous frame

...until the mouse turns around in two frames

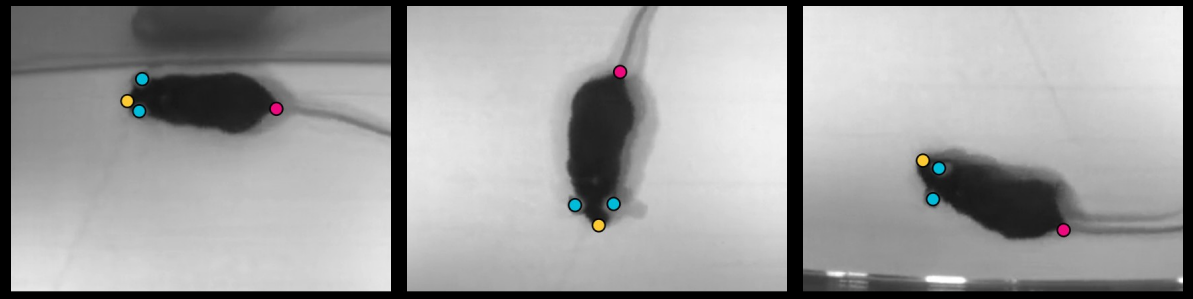
3. Look ahead of the direction
of motion

...until it rears or walks backwards

every fix needs another fix

Let the Model Find the Procedure

We can recognise correct answers, so we collect them and let a model learn the rest



examples: frames where a person clicked on the snout, ears and tail base

learning → a model

...and a trained model at work, on a video that nobody labelled



DeepLabCut open-field example, via Ye *et al.* (2024), CC BY 4.0

on Thursday: how DeepLabCut does exactly this, for real

What Is Learning?

Programs, data, and the map of machine learning

Programming vs. Learning

In classical programming, we write the rules

Data + Rules → Answers

written by you

computed

what we put in

what we get out

Programming vs. Learning

In machine learning, we give the answers and get the rules



curator's corner: the answers we feed in had better be right

Learning, Defined

When has a program learned? When it does a task better with experience.

A program learns from experience E at a task T if its performance P at T improves with E.

T find the snout in a frame

P distance from the true snout, in pixels

E frames where a person clicked on the snout

Mitchell (1997)

A Frame Is a Row of Numbers

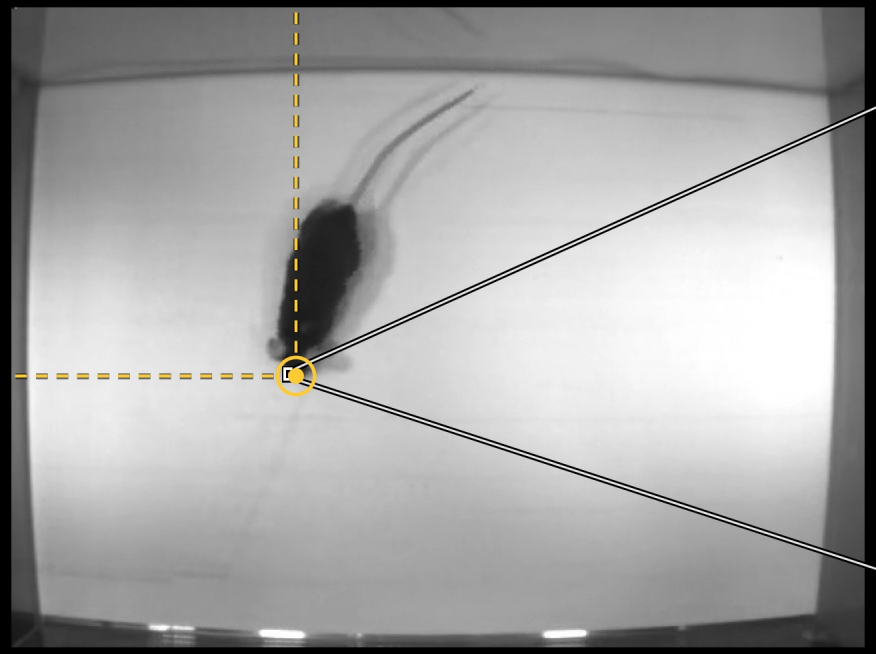
To a computer, a frame is 307,200 numbers, and a click is two more



Video: DeepLabCut open-field example, via Ye *et al.* (2024), CC BY 4.0

A Frame Is a Row of Numbers

To a computer, a frame is 307,200 numbers, and a click is two more



94	62	45	37	34	30	29	30
118	75	45	34	33	29	29	29
128	87	56	37	34	28	30	31
151	118	85	52	43	35	34	36
164	142	119	80	61	49	42	40
173	156	143	119	91	72	61	51
183	171	167	150	121	94	76	65
186	183	183	178	153	126	101	83

row 1 | row 2 | row 3 | the click

… 94 62 45 37 34 30 29 30 | 118 75 45 34 33 29 29 29 | 128 87 56 37 34 28 30 31 … | 213 276

$$\mathbf{x} = (x_1,\ x_2,\ \ldots,\ x_{307{,}200}) \qquad \mathbf{y} = (x_{\text{snout}},\ y_{\text{snout}})$$

n frames → a table, $n \times 307{,}200$, plus $n \times 2$ labels

Video: DeepLabCut open-field example, via Ye *et al.* (2024), CC BY 4.0

Data as a Matrix

Almost every dataset this week is a table: one row per sample, one column per feature

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix} \qquad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Data as a Matrix

Almost every dataset this week is a table: one row per sample, one column per feature

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix} \qquad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

a row is a sample: one trial, one image, one patient

Data as a Matrix

Almost every dataset this week is a table: one row per sample, one column per feature

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix} \qquad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

a row is a sample: one trial, one image, one patient

a column is a feature: one neuron, one pixel, one measurement

Data as a Matrix

Almost every dataset this week is a table: one row per sample, one column per feature

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix} \qquad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

a row is a sample: one trial, one image, one patient

a column is a feature: one neuron, one pixel, one measurement

y holds one label per row: what we want to predict (the snout needs two)

$$X \in \mathbb{R}^{n \times d}, \qquad y \in \mathbb{R}^n$$

curator's corner: someone decided what a row is, what a column is, and who wrote the labels

The Map of Machine Learning

Two big families, depending on whether we have the answers

Supervised

we have the answers: pairs (x, y)

Monday to Thursday morning

- **regression** — predict a number
- **classification** — predict a category

Unsupervised

we only have x

Thursday

- **clustering** — find groups
- **dimensionality reduction** — find the few directions that matter

also: self-supervised and reinforcement learning, not this week

Which Kind of Task?

Your turn: shout it out

Tomorrow's temperature in Trieste	*regression*
Is this email spam?	*classification*
Group neurons by how they respond	*clustering*
Draw 11,000 neurons as one 2-D picture	*dimensionality reduction*
Where is the snout in this frame?	*regression: two numbers*

Start Small

307,200 inputs cannot be drawn; one input and one output can

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix} \qquad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

the snout: 307,200 numbers in, 2 numbers out

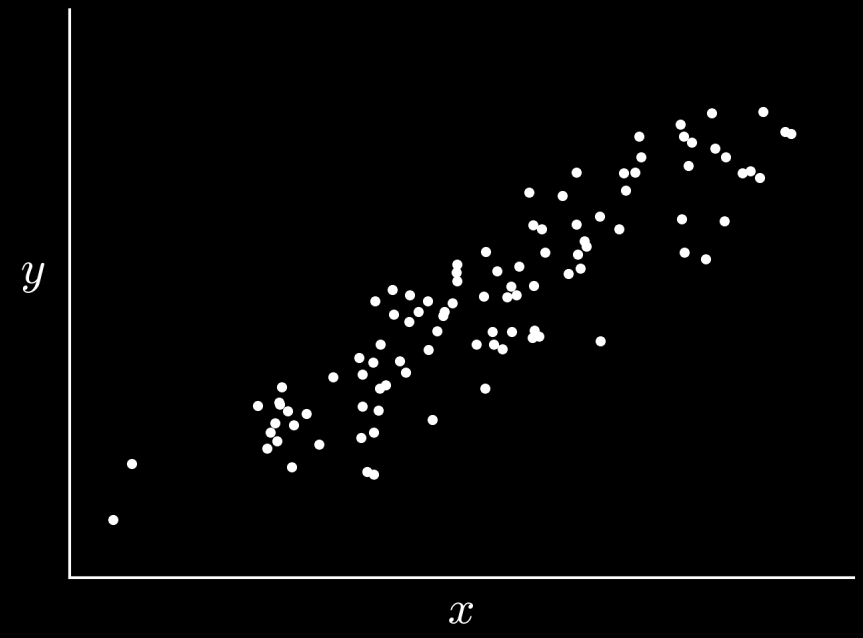
Start Small

307,200 inputs cannot be drawn; one input and one output can

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix} \qquad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

the snout: 307,200 numbers in, 2 numbers out

keep one feature and one label



each row is now a point we can draw

Every idea we need fits on this plot, and carries over to the snout unchanged

How Wrong Am I?

Turning “a good model” into a number we can optimise

A Model Is a Function with Knobs

It turns inputs into predictions; its parameters are knobs we can turn

$$\hat{y} = f(x;\, \theta)$$

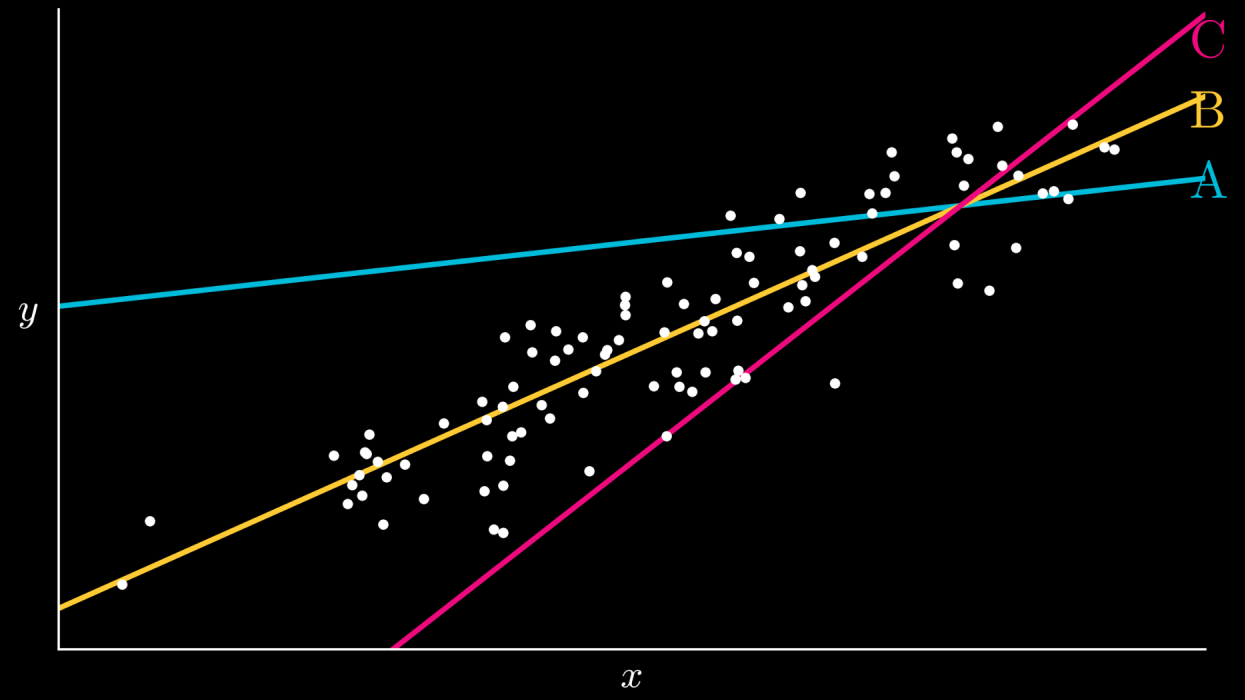
$\hat{y}$	x	θ
the prediction	the input	the knobs

The simplest model, a straight line, has two knobs:

$$\hat{y} = w\,x + b$$

Which Line Is Best?

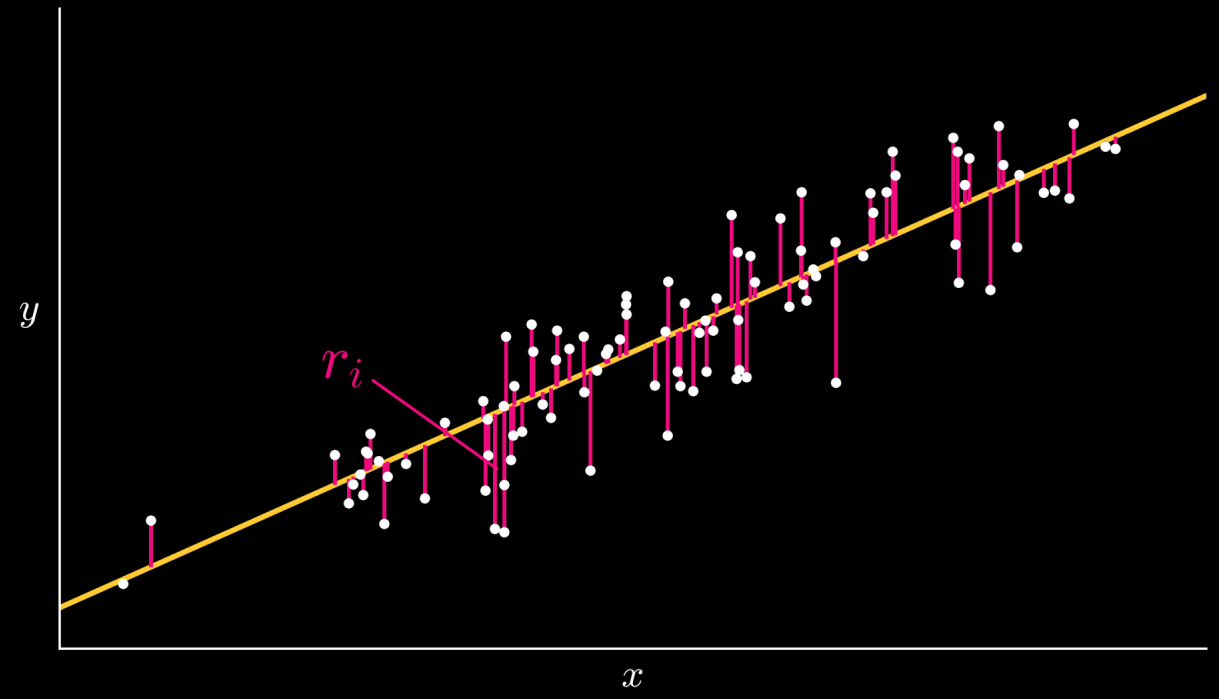
Your turn: three settings of the knobs. Which line would you keep?



To choose, we need a number that says *how wrong* each line is

The Loss

Measure the error on every example, then average



$$r_i = y_i - \hat{y}_i$$

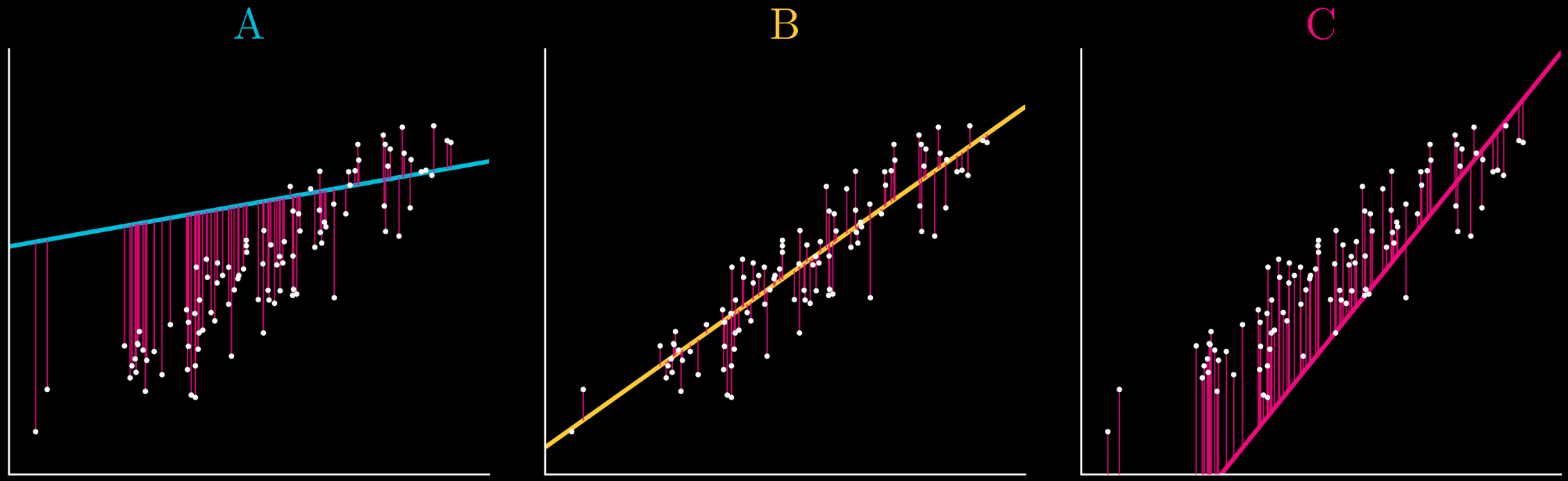
$$L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \big(y_i - f(x_i; \theta)\big)^2$$

the mean squared error

for the snout, the residual is a distance in pixels: Mitchell's P

The Verdict

One number per line, and it agrees with your eyes

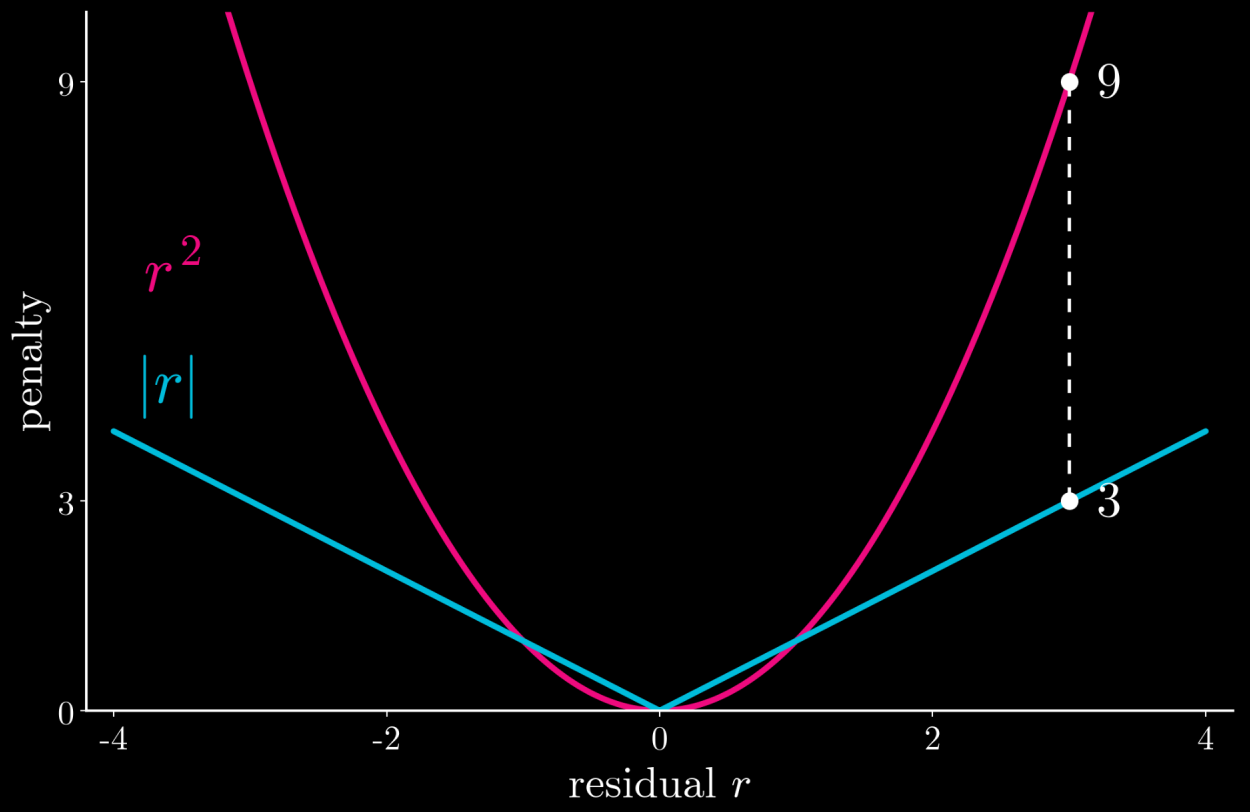


$L = 6.87$ $L = 0.92$ $L = 6.18$

B is the line the data came from, and even it scores 0.92, not 0: the noise

Squared or Absolute?

Different losses have different opinions about mistakes

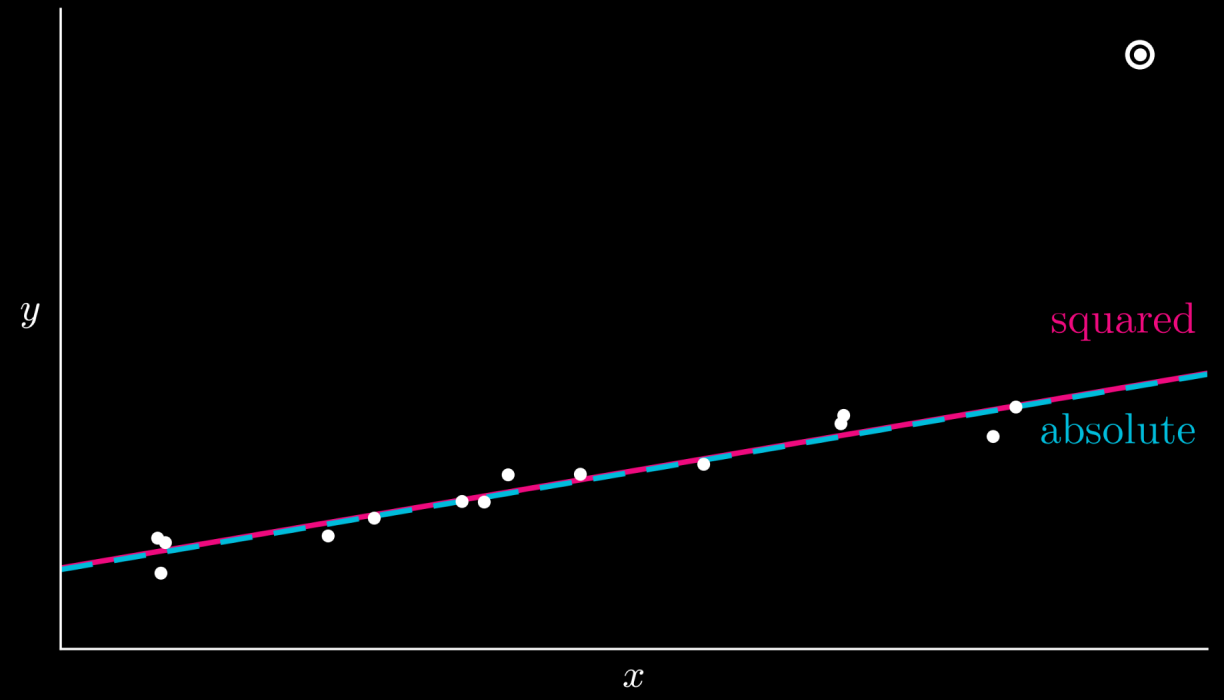


A mistake of 3 costs 3 under $|r|$, but 9 under r^2.

The squared loss *hates* big mistakes.

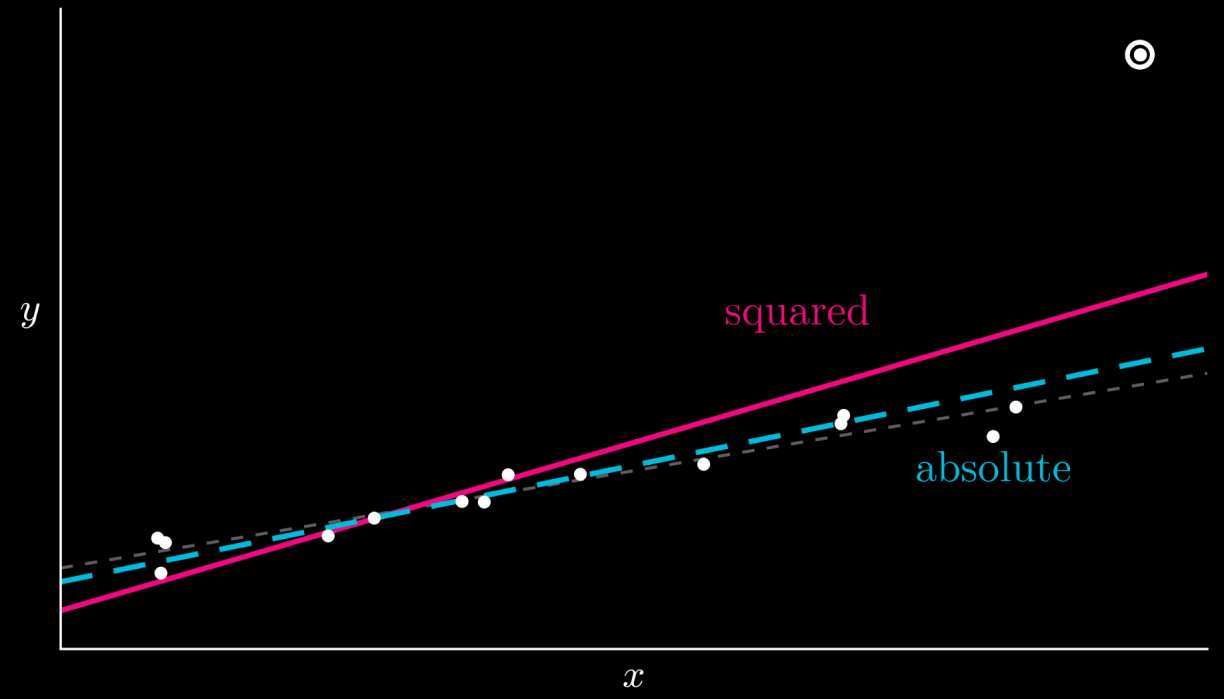
One Bad Measurement

Your turn: one value is corrupted. Which fit moves more?



One Bad Measurement

Your turn: one value is corrupted. Which fit moves more?



curator's corner: one bad value can move the whole model; look at your data first

Learning = Minimising the Average Loss

This one line is most of machine learning

$$\theta^\star = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \ell\big(y_i,\ f(x_i;\theta)\big)$$

the optimiser — the data — the loss — the model

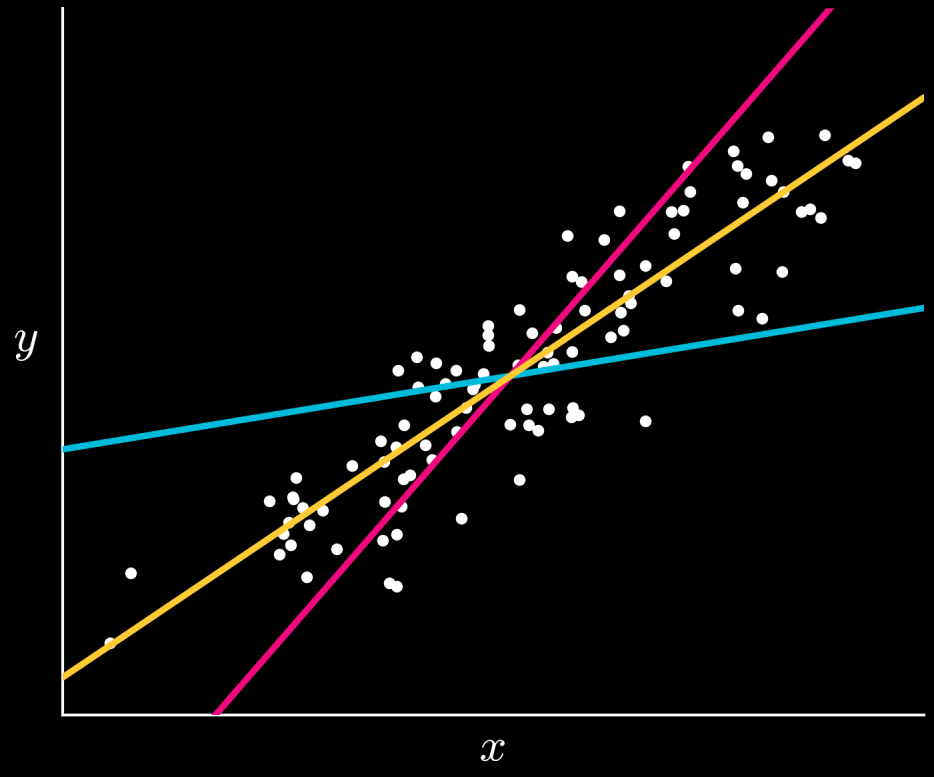
a.k.a. empirical risk minimisation

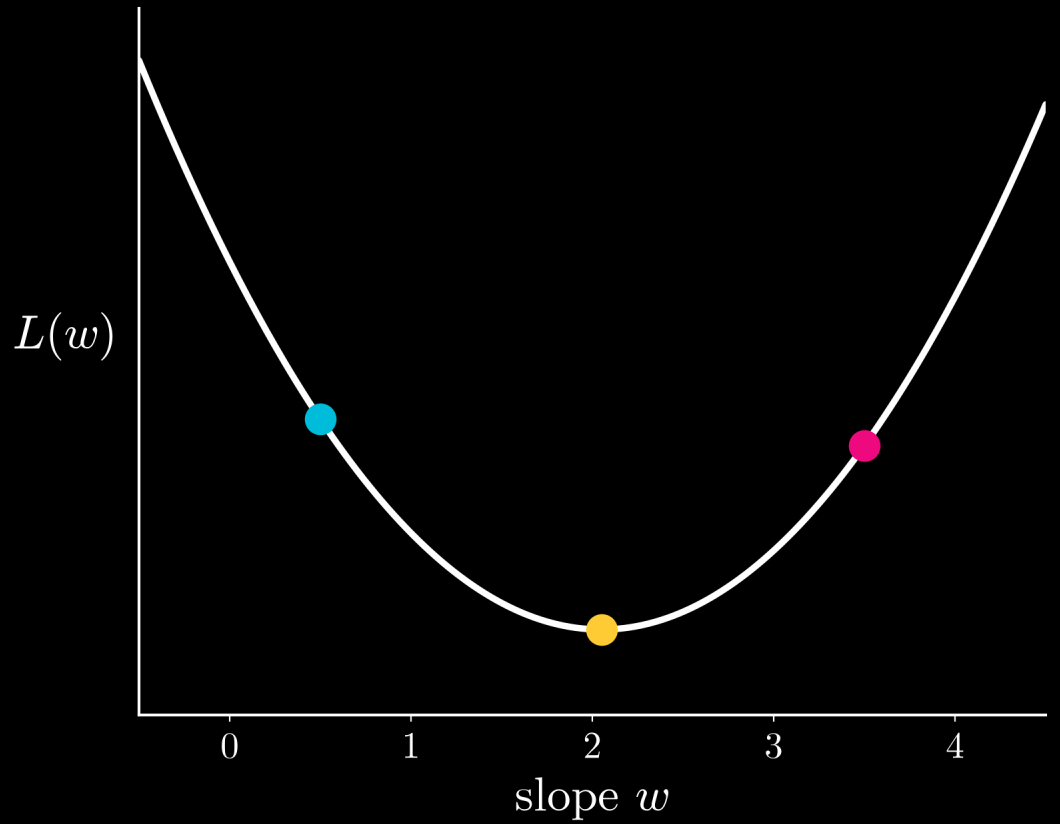
Rolling Downhill

How to find the knobs that minimise the loss

The Loss Landscape

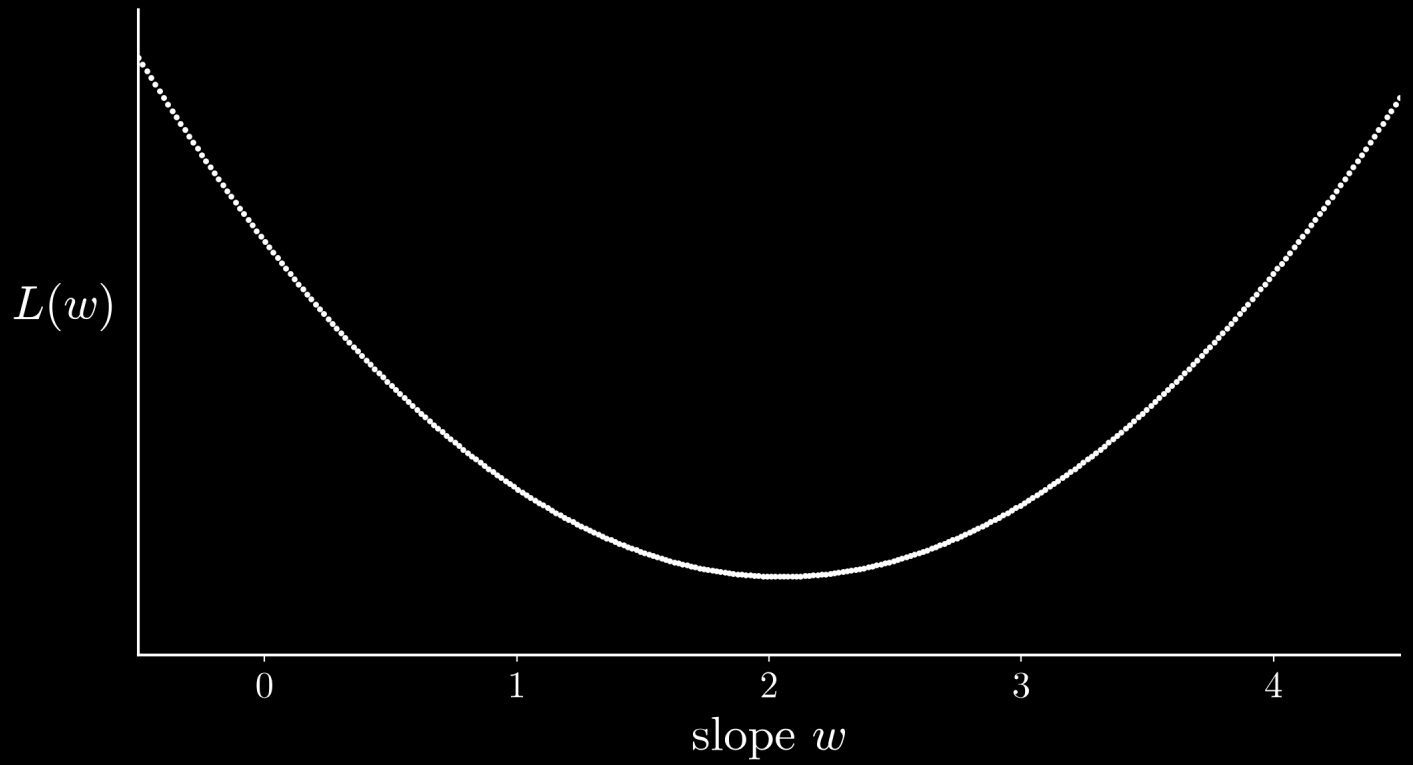
Every setting of the knobs has a loss: together they form a landscape





Why Not Try Every Line?

Your turn: how many losses would brute force need?



one knob, 100 values:
100 losses

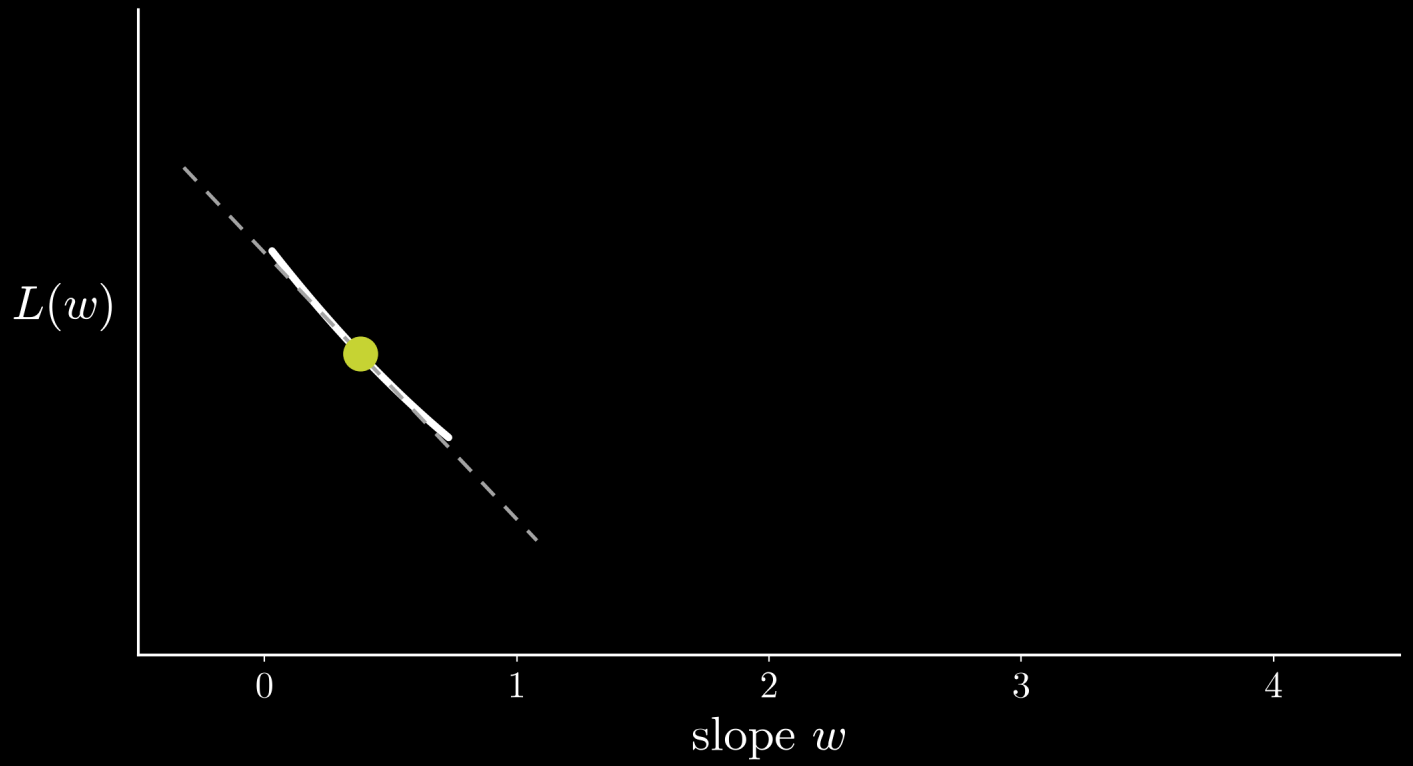
two knobs, w and b:
$100 \times 100 = 10{,}000$ losses

a network that finds the snout:
about 24 million knobs

$$100^{24\,000\,000} = 10^{48\,000\,000}$$

Why Not Try Every Line?

Your turn: how many losses would brute force need?



All we can afford: the loss and its slope, where we stand

one knob, 100 values:
100 losses

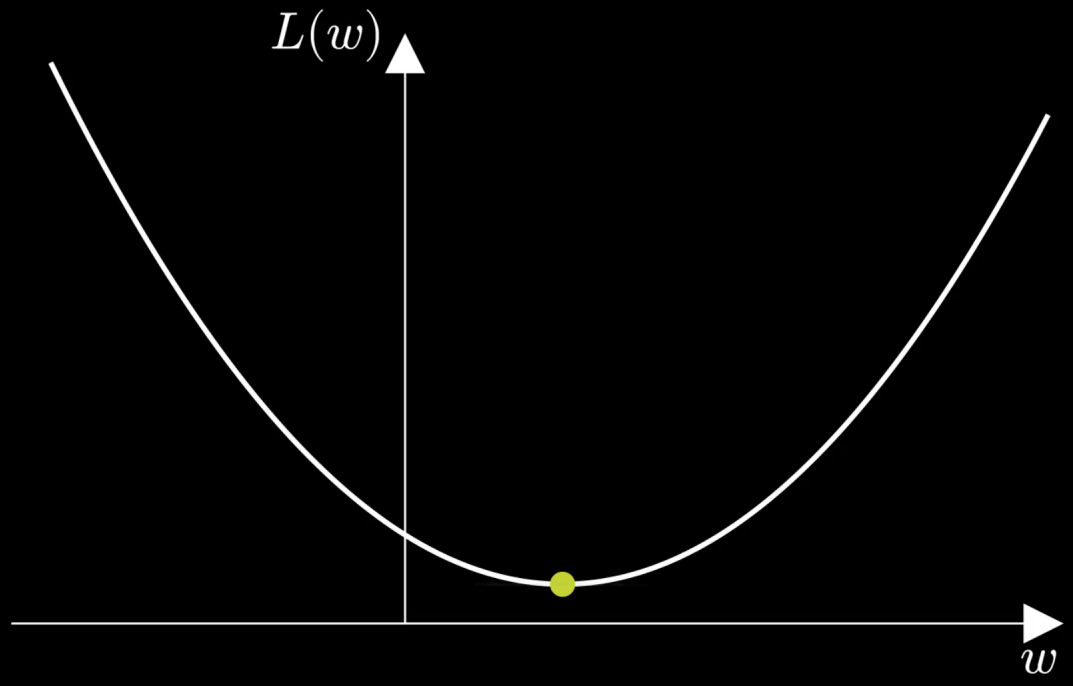
two knobs, w and b:
$100 \times 100 = 10{,}000$ losses

a network that finds the snout:
about 24 million knobs

$$100^{24\,000\,000} = 10^{48\,000\,000}$$

Gradient Descent

Look at the slope under your feet, take a small step downhill, repeat

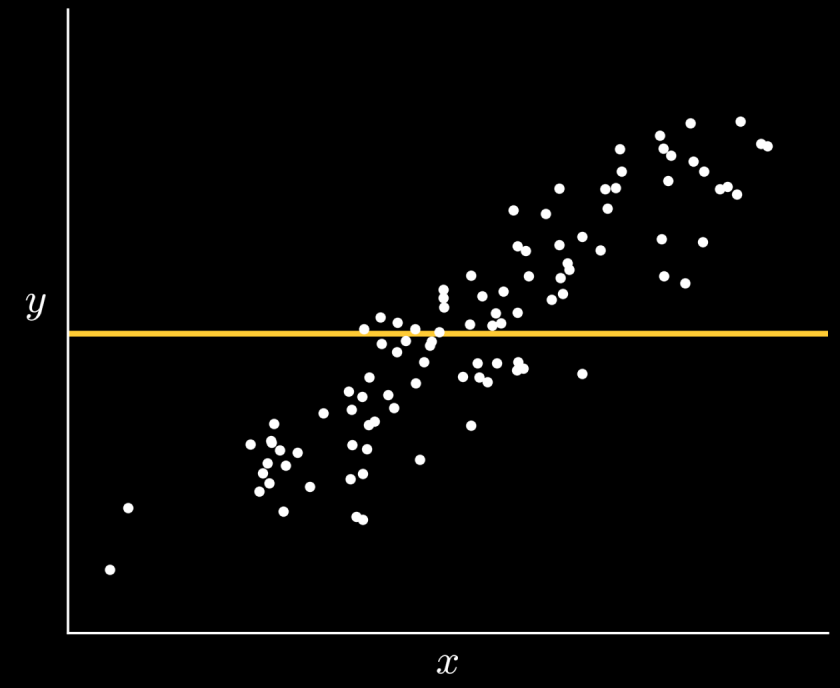


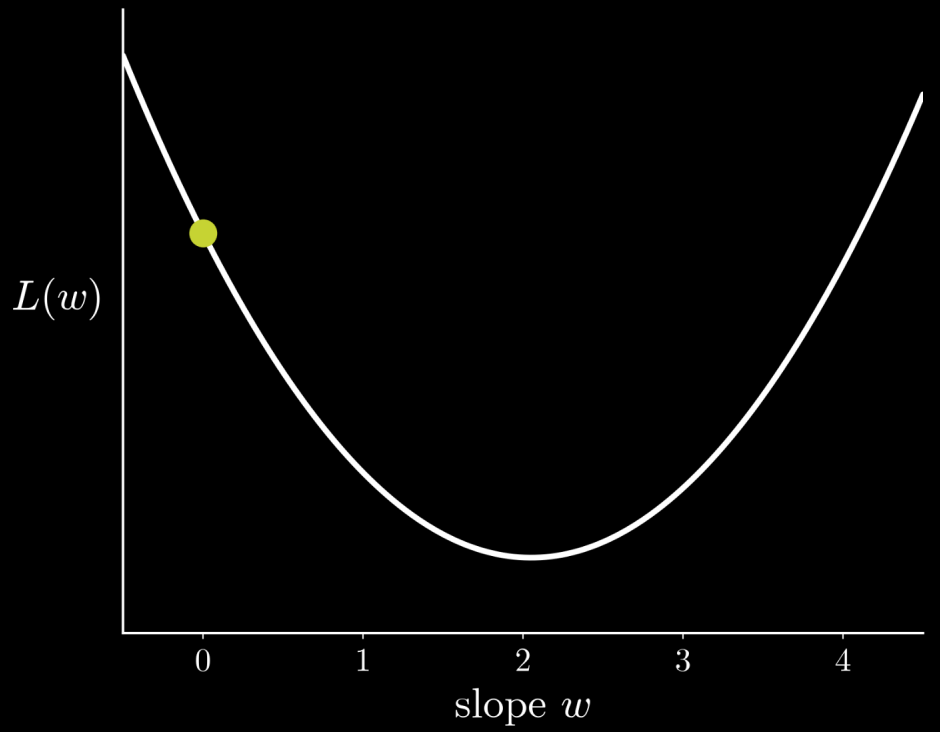
$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

η is the learning rate: the step size

One Step, by Hand

The slope of the loss is an average of pushes, one per example



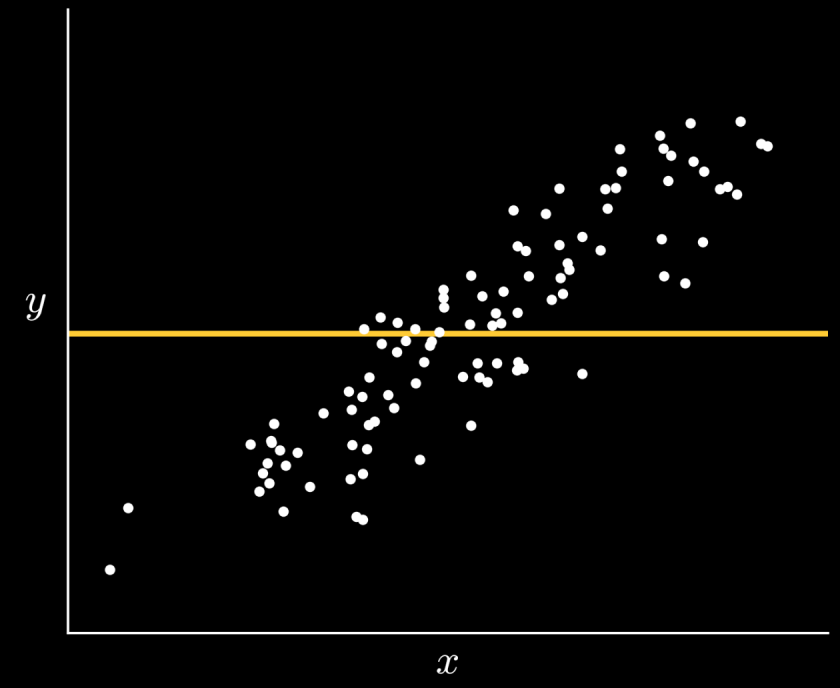


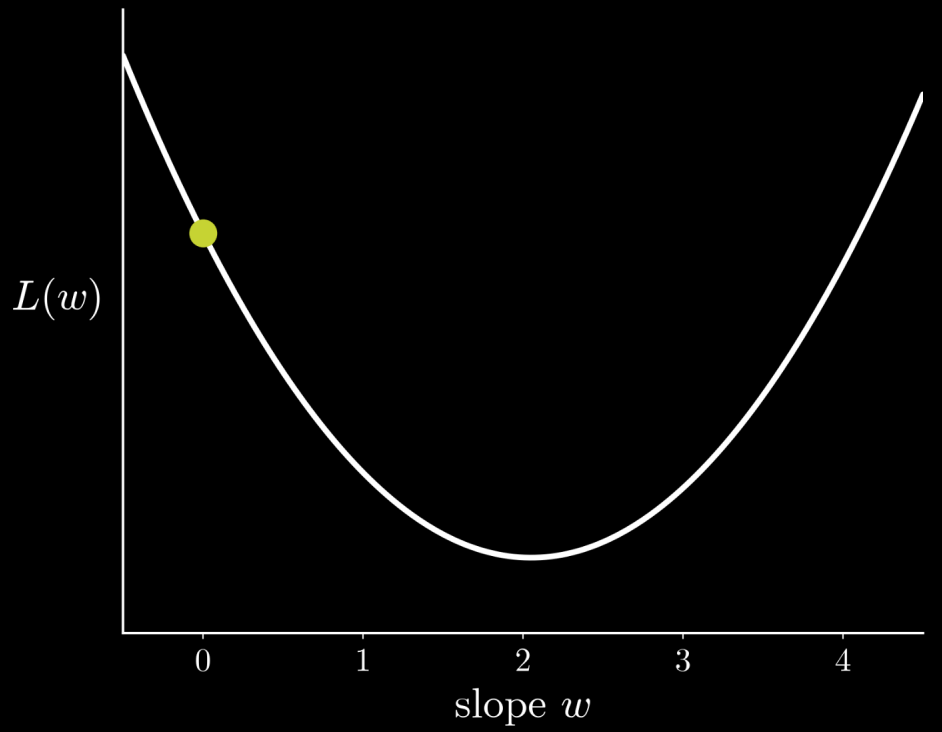
$$\frac{\partial L}{\partial w} = -\frac{2}{N} \sum_i x_i \left(y_i - \hat{y}_i \right)$$

each point pulls the slope towards itself, in proportion to its residual

One Step, by Hand

The slope of the loss is an average of pushes, one per example



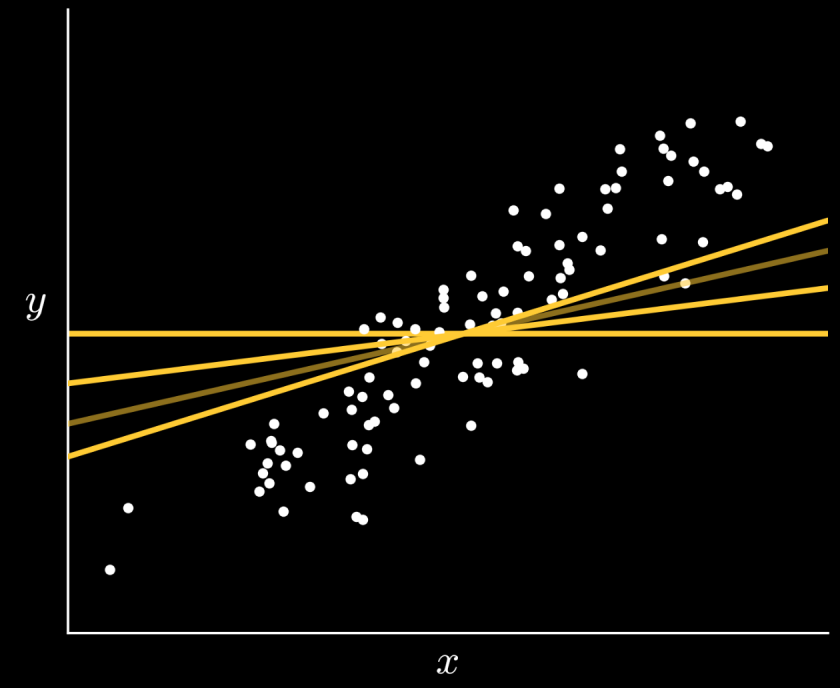


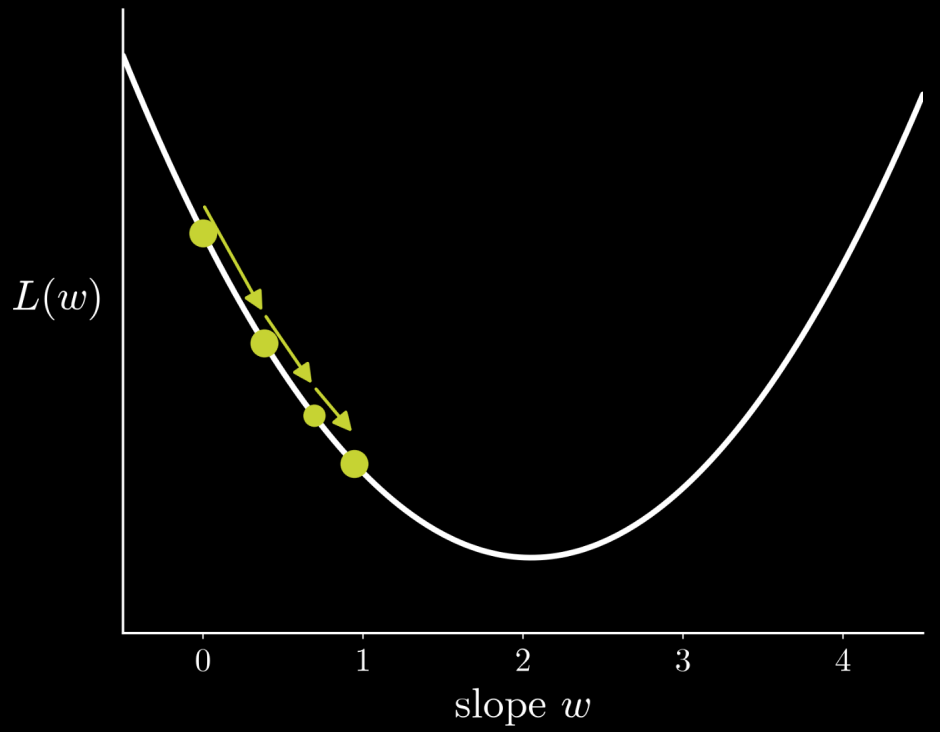
$$\frac{\partial L}{\partial w} = -\frac{2}{N} \sum_i x_i \left(y_i - \hat{y}_i \right)$$

at $w = 0$ it is -3.82: downhill is to the right

One Step, by Hand

The slope of the loss is an average of pushes, one per example





$$\frac{\partial L}{\partial w} = -\frac{2}{N}\sum_i x_i \left(y_i - \hat{y}_i\right)$$

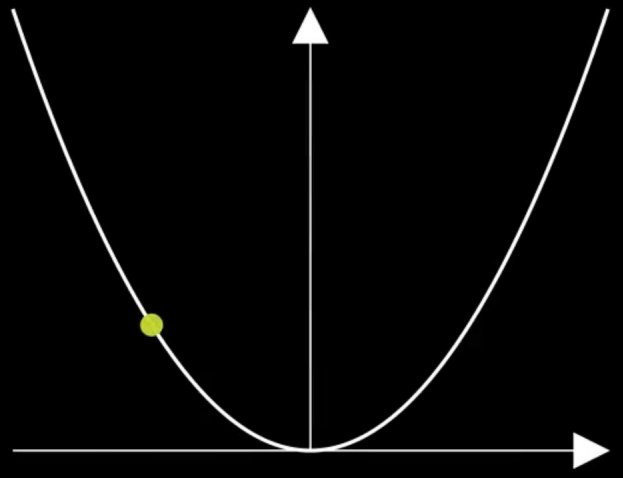
$$w \leftarrow 0 - 0.1 \times (-3.82) = 0.38$$

$\eta = 0.1$ is a cautious step. Why not bigger ones?

What If the Step Is Too Big?

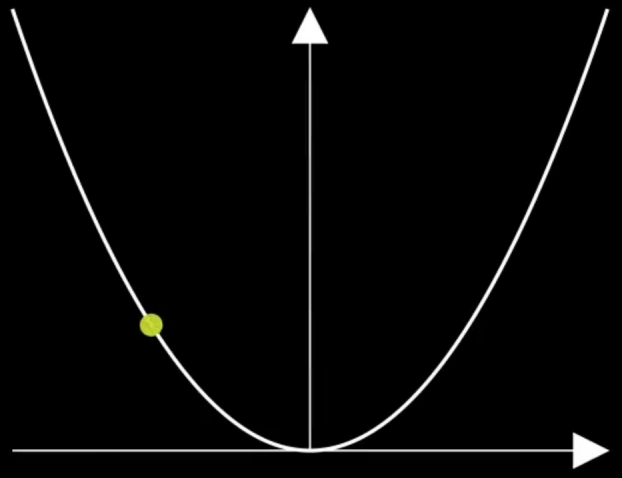
Your turn: same landscape, three step sizes. What happens to each?

$\eta = 0.05$



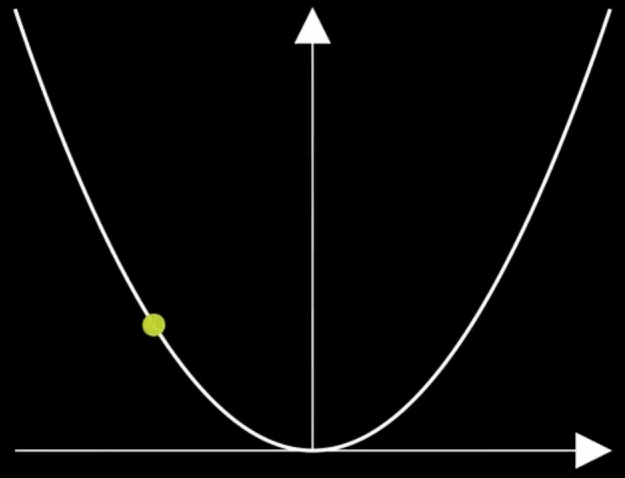
too small: it crawls

$\eta = 0.45$



about right

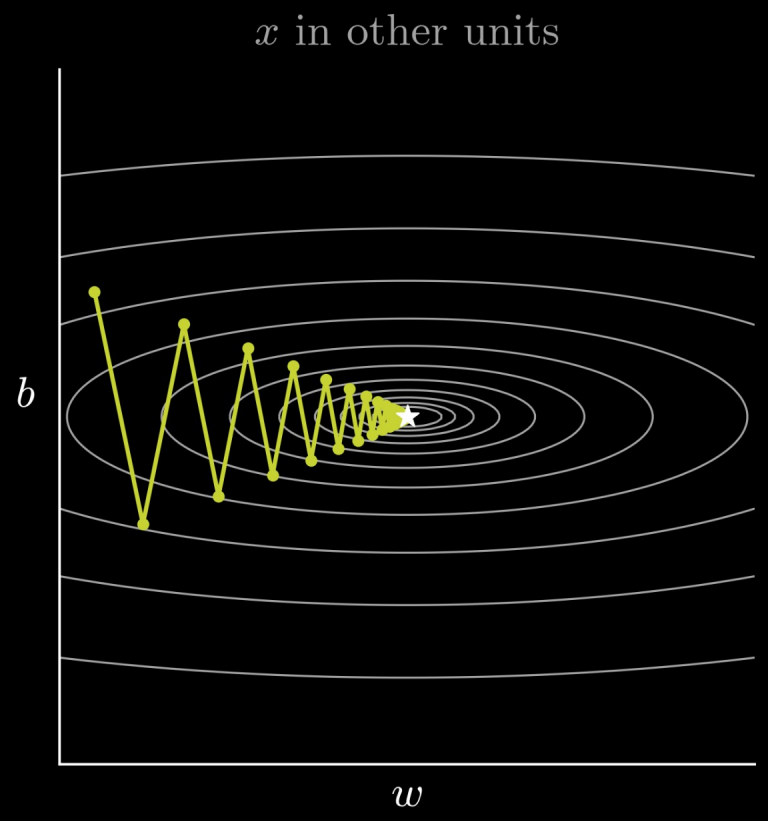
$\eta = 1.1$

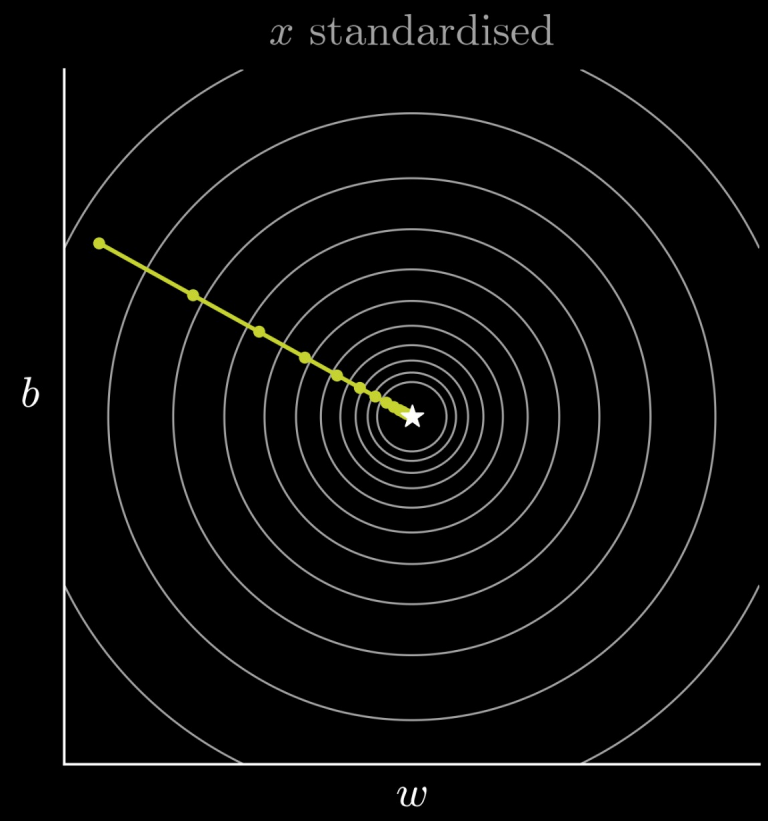


too big: it diverges

Too Big and Too Small at Once

On a stretched valley, one step size cannot suit both directions





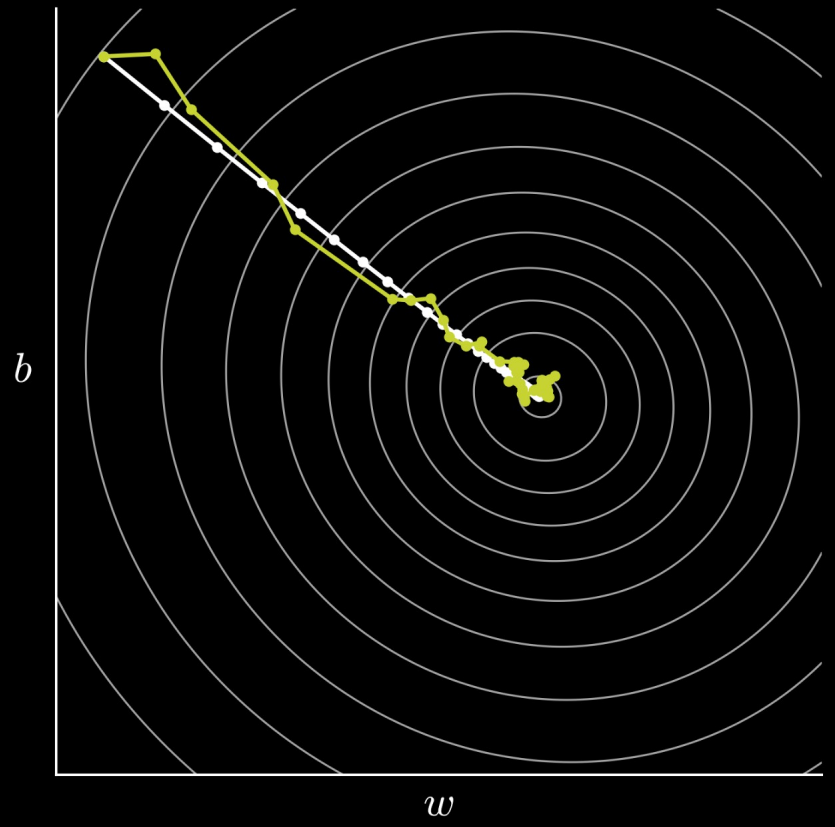
across the valley: too big, it bounces

along the valley: too small, it crawls

curator's corner: standardise your features

Mini-Batches

The slope is an average over all examples; estimate it from a random handful



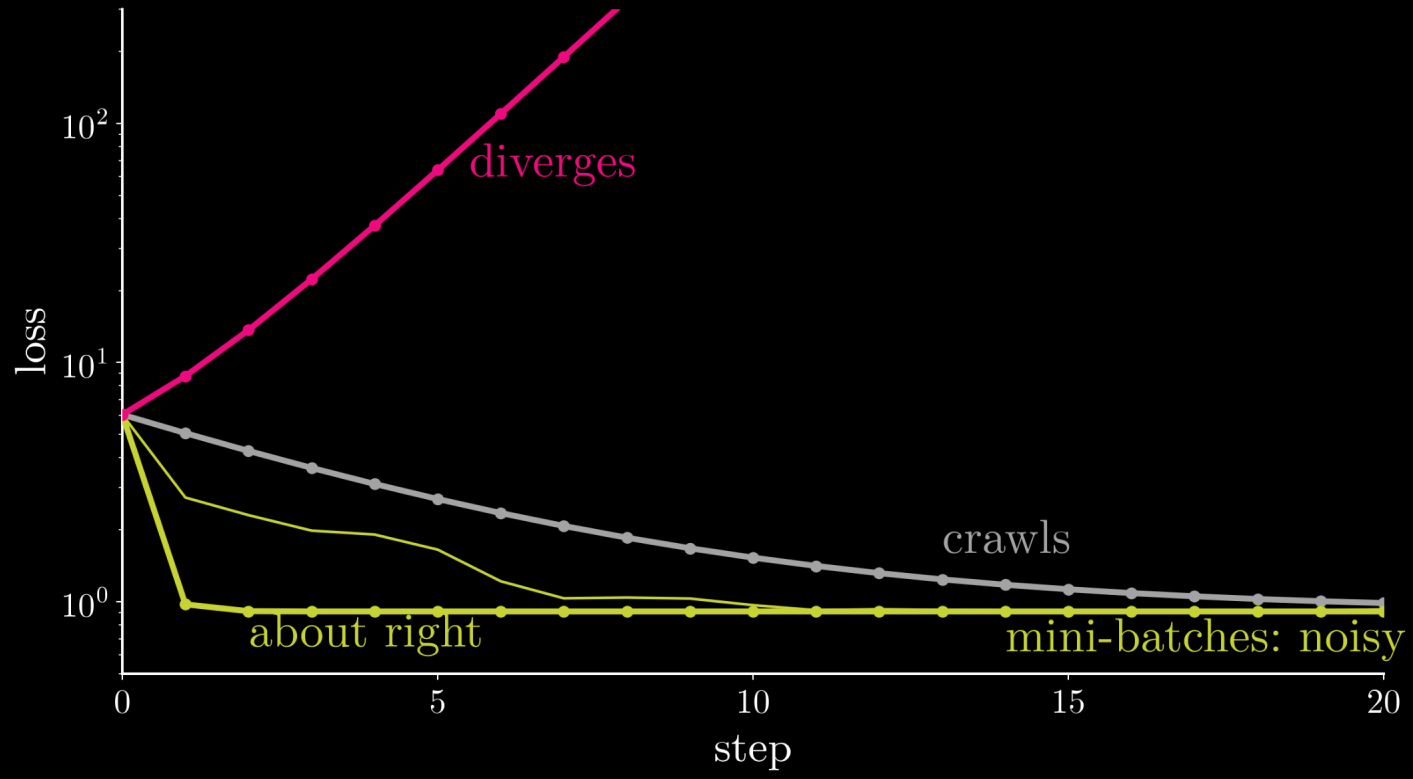
white: all 100 examples, at every step

lime: four random examples per step.
Cheaper, noisier, same destination.

this is how every neural network is trained: Alessio will take it from here

Watch the Loss

With millions of knobs you cannot watch the ball, but you can always watch the loss



The floor is not zero: 0.91, the noise

The Recipe

This morning, in four lines

Data	$(x_i,\ y_i)$	our cloud of 100 points
Model	$\hat{y} = w\,x + b$	a line: two knobs
Loss	the squared error	
Optimiser	gradient descent, step size η	

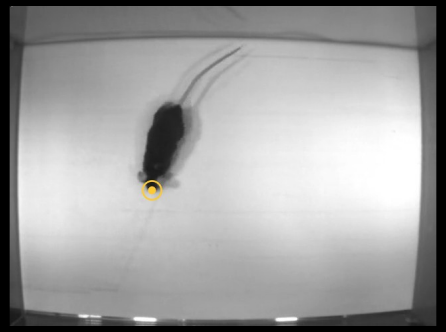
curator's corner: every warning this morning lived in the Data slot

The Snout, as a Recipe

We can already write three of the four lines

Data $X \in \mathbb{R}^{n \times 307\,200}$ $Y \in \mathbb{R}^{n \times 2}$

frames, and where a person clicked the snout



Model ?

Loss squared distance, in pixels, to the clicked snout

Optimiser gradient descent, in mini-batches

Thursday: what goes in the Model slot

Break

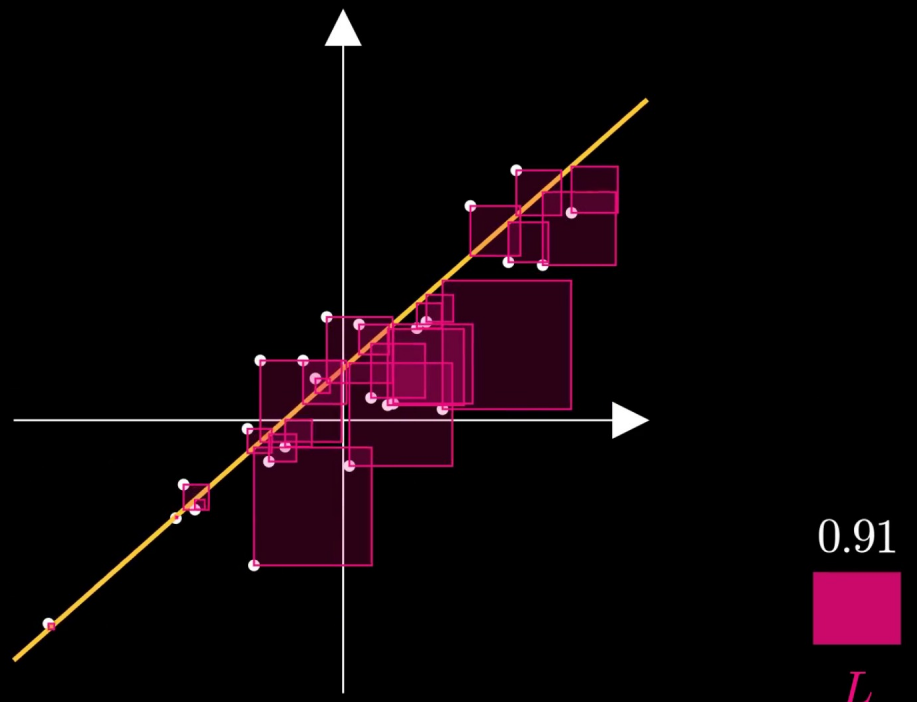
Next: our first two models, and your first notebook

Our First Two Models

One model you already know, and one for yes-or-no answers

Linear Regression

You already know it: a line, the squared error, and rolling downhill



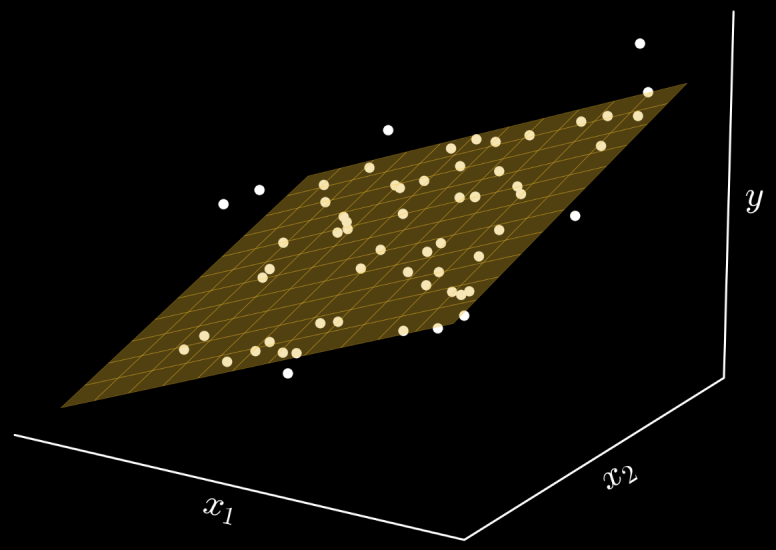
$$\hat{y} = w\,x + b \qquad L = \frac{1}{N}\sum_i (y_i - \hat{y}_i)^2$$

for this model, the minimum can also be written down exactly:

$$\theta^\star = (X^\top X)^{-1} X^\top y$$

More Features, Same Idea

With d features the line becomes a plane, or a hyperplane



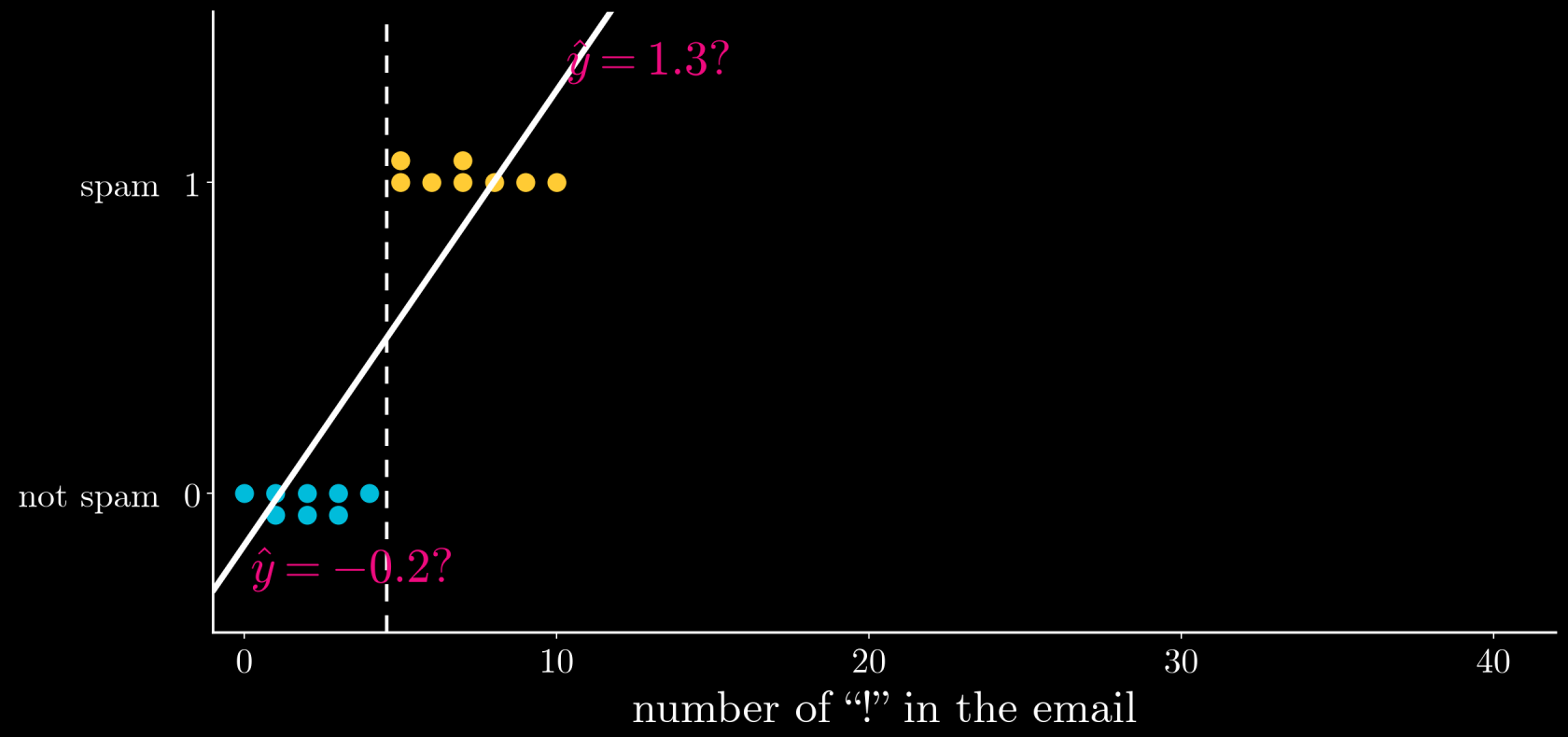
one weight per feature:

$$\hat{y} = w_1 x_1 + \cdots + w_d x_d + b = \mathbf{w}^\top \mathbf{x} + b$$

add x^2, x^3, … as features and the line can bend: tomorrow's overfitting story

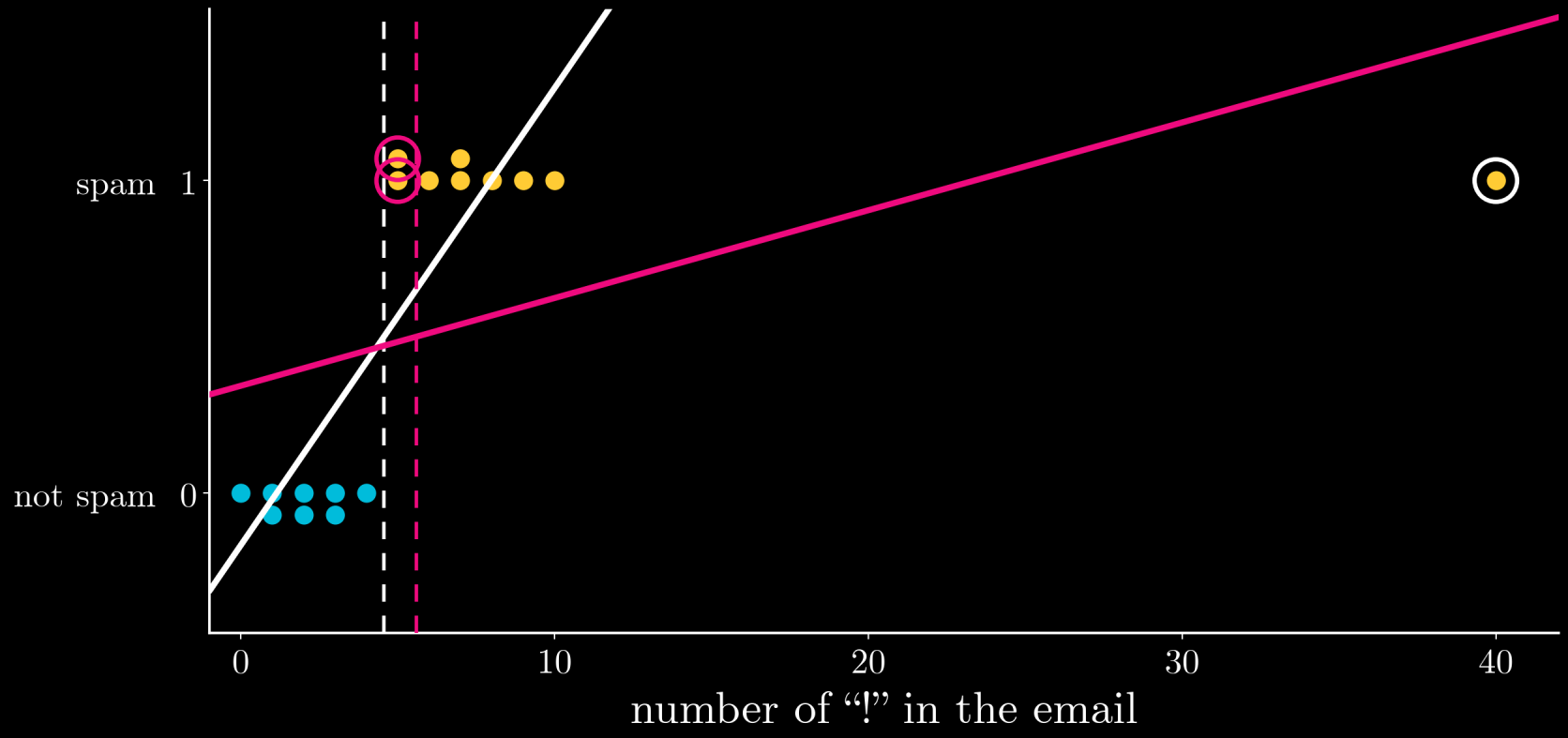
When the Answer Is Yes or No

Your turn: fit our line to yes-or-no answers. What could go wrong?



When the Answer Is Yes or No

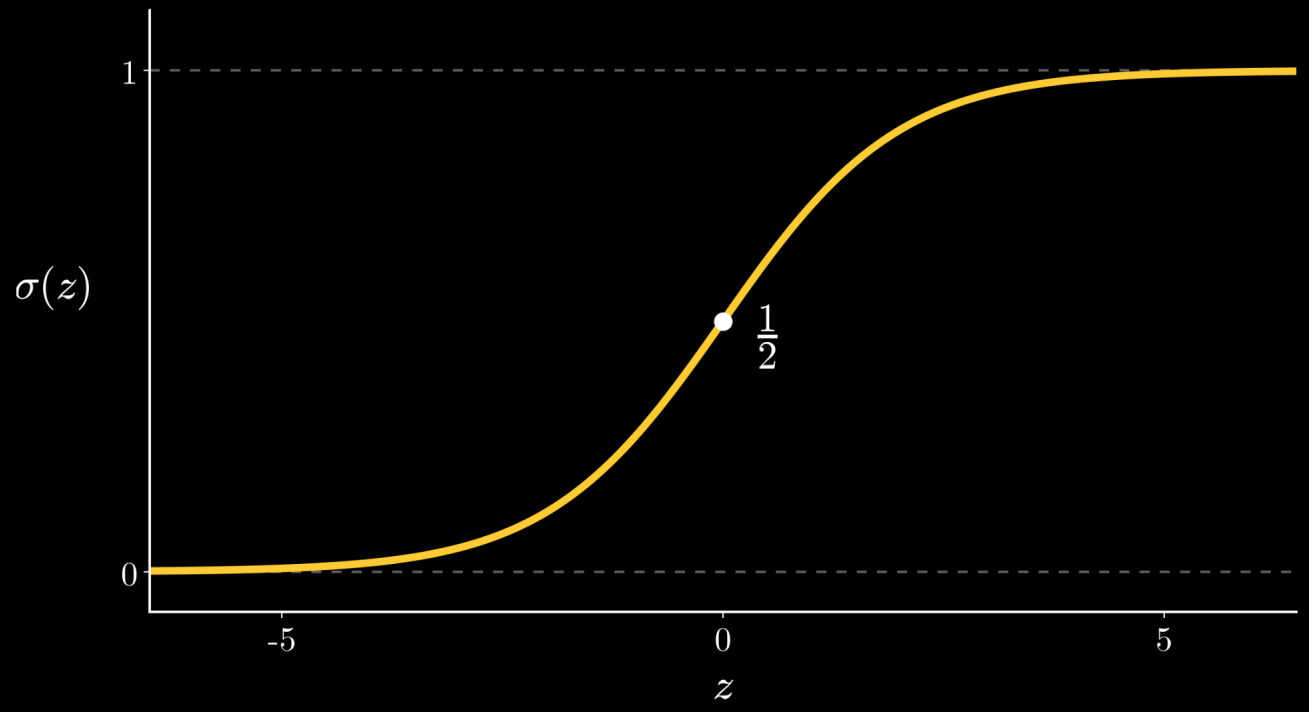
Your turn: fit our line to yes-or-no answers. What could go wrong?



We need an output between 0 and 1, and a loss that never punishes being right

From Lines to Probabilities

To classify, squash the line into a probability



squash any number into (0, 1):

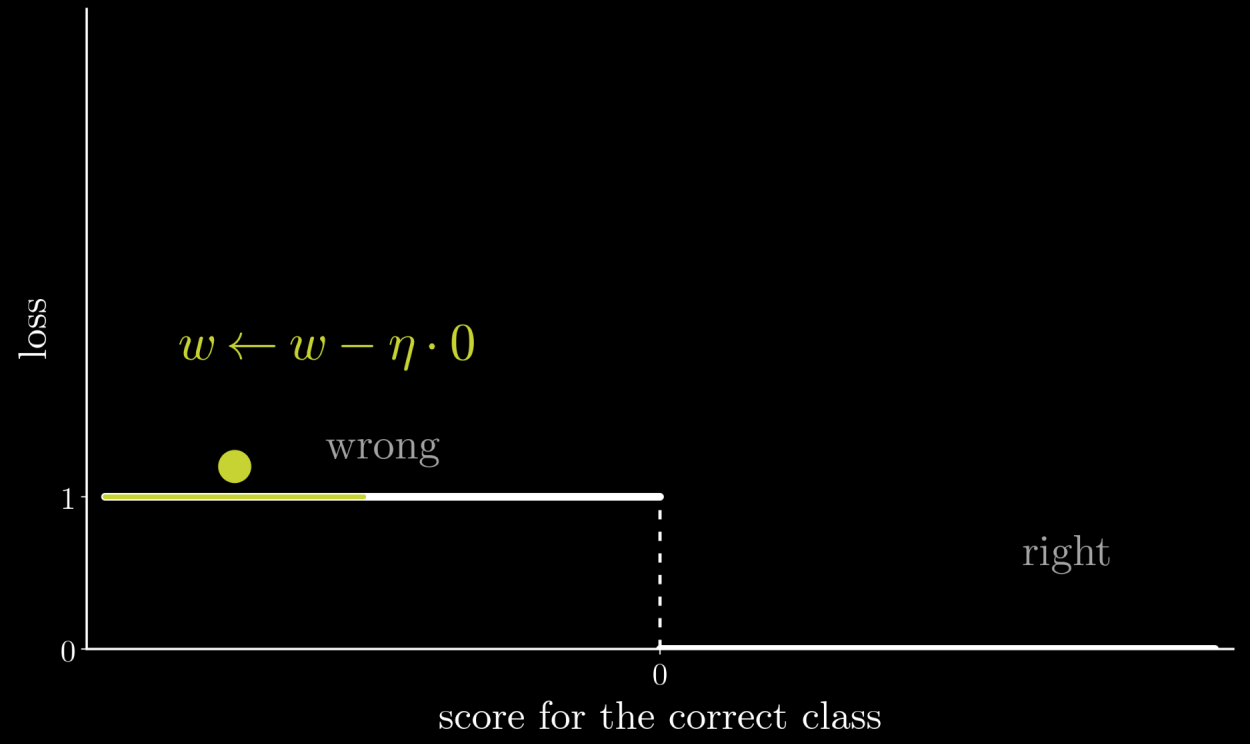
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

the model:

$$p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

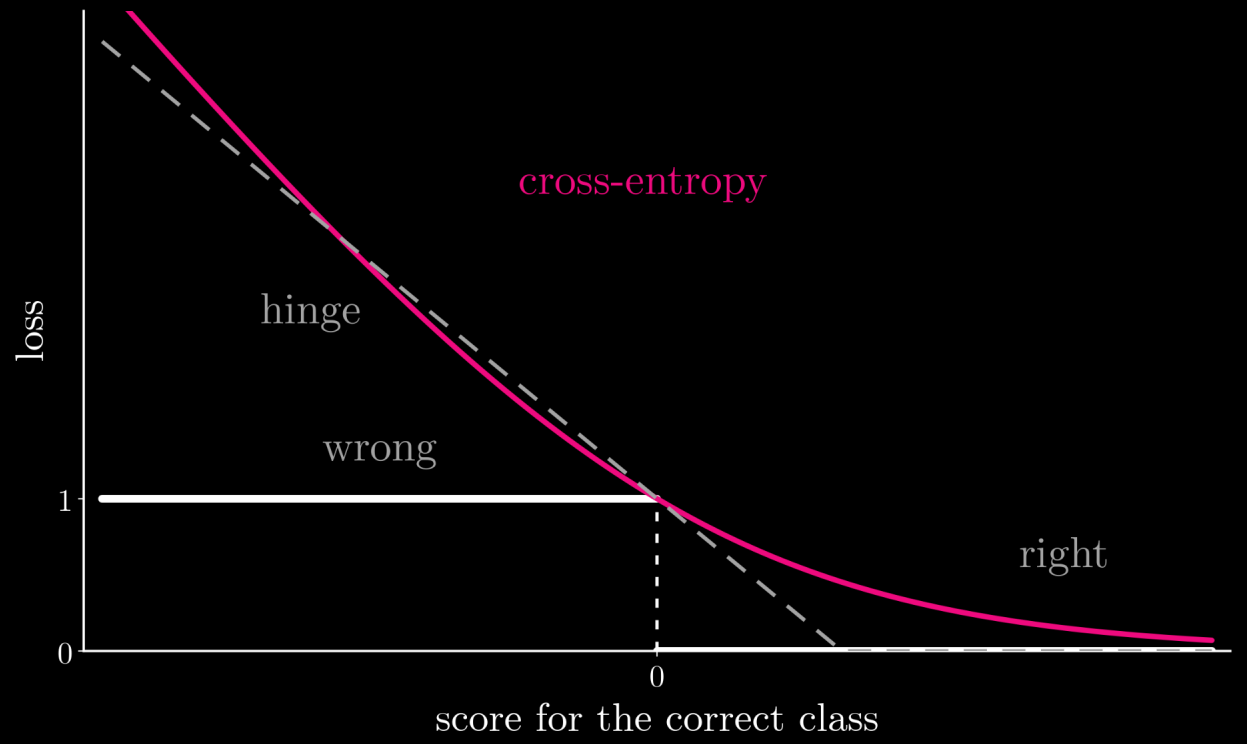
Counting Mistakes Is Not Enough

On flat ground, gradient descent has nowhere to roll



Counting Mistakes Is Not Enough

On flat ground, gradient descent has nowhere to roll



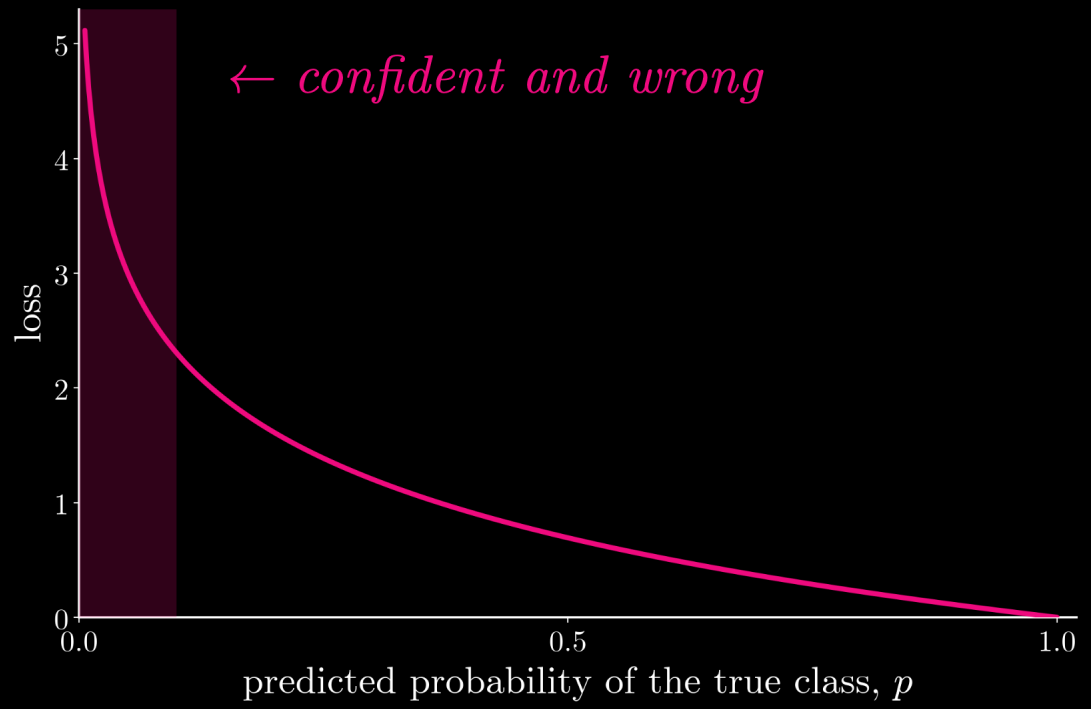
We minimise what we can roll down; we report what we care about

the hinge loss comes back on Thursday, with SVMs

Cross-Entropy

Your turn: the answer is yes. What should a guess of p cost?

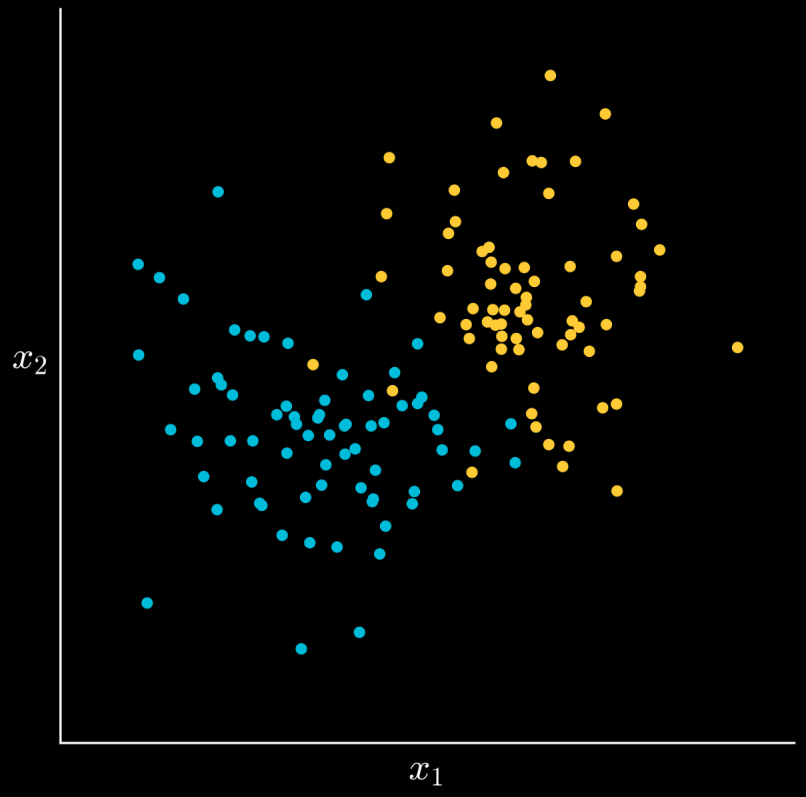
$$\ell(y, p) = -\big[\, y \log p + (1 - y) \log(1 - p) \,\big]$$



curator's corner: a wrong label looks like a confident mistake, and costs the most

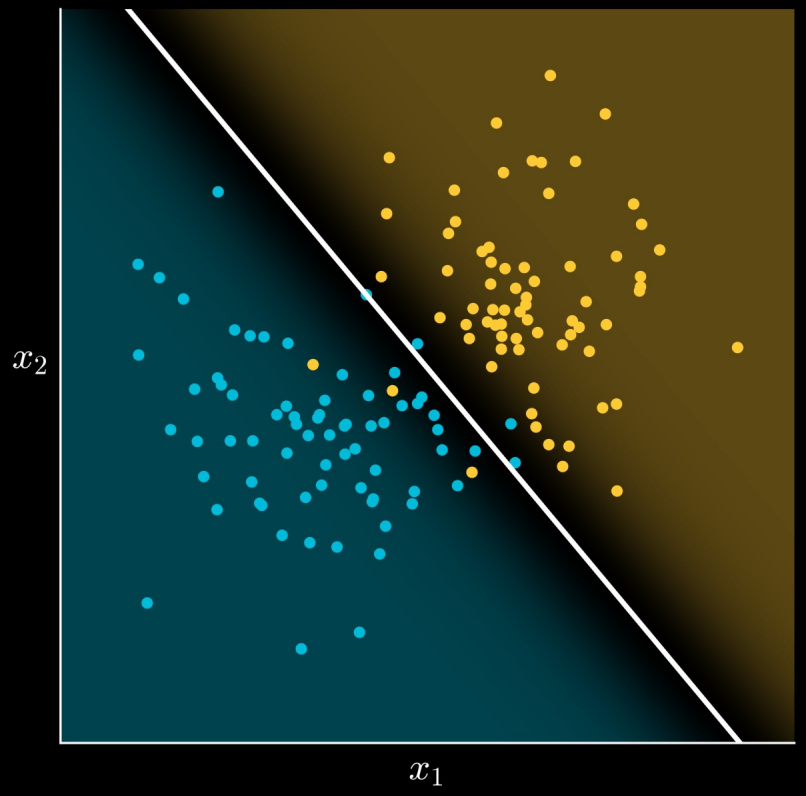
The Decision Boundary

Where the probability crosses one half, the model changes its mind



The Decision Boundary

Where the probability crosses one half, the model changes its mind



shading: the predicted probability of yellow

the boundary, where it equals one half:

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

on Thursday: data that no straight line can separate

Logistic Regression as a Recipe

Same card, new ingredients: only the model and the loss change

Data	$(x_i,\ y_i)$	our cloud
Model	$\hat{y} = w\,x + b$	a line
Loss		the squared error
Optimiser		gradient descent

Logistic Regression as a Recipe

Same card, new ingredients: only the model and the loss change

Data	$(x_i,\ y_i \in \{0, 1\})$	labels are 0 or 1
Model	$\hat{y} = w\,x + b$	a line
Loss	the squared error	
Optimiser	gradient descent	

Logistic Regression as a Recipe

Same card, new ingredients: only the model and the loss change

Data	$(x_i,\ y_i \in \{0, 1\})$	labels are 0 or 1
Model	$\sigma(\mathbf{w}^\top \mathbf{x} + b)$	a line, squashed into a probability
Loss	the squared error	
Optimiser	gradient descent	

Logistic Regression as a Recipe

Same card, new ingredients: only the model and the loss change

Data	$(x_i,\ y_i \in \{0, 1\})$	labels are 0 or 1
Model	$\sigma(\mathbf{w}^\top \mathbf{x} + b)$	a line, squashed into a probability
Loss	cross-entropy	
Optimiser	gradient descent	

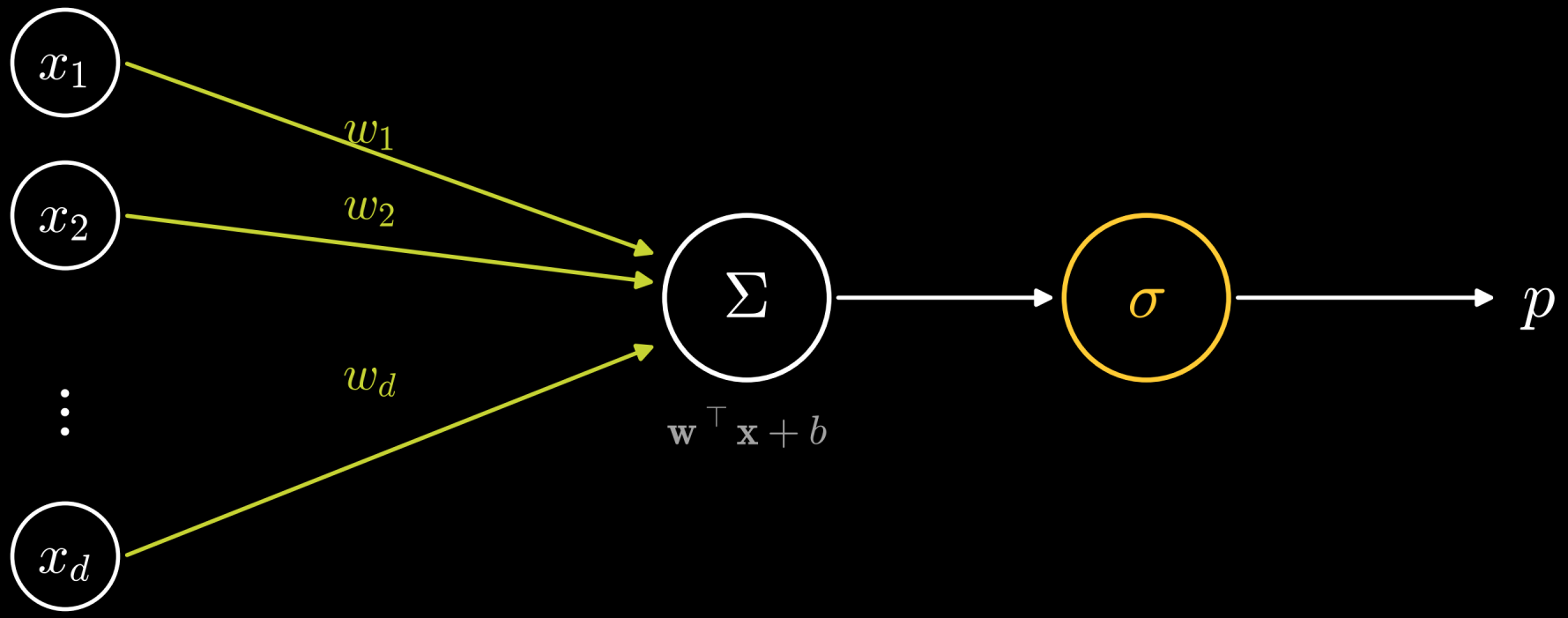
Logistic Regression as a Recipe

Same card, new ingredients: only the model and the loss change

Data	$(x_i,\ y_i \in \{0, 1\})$	labels are 0 or 1
Model	$\sigma(\mathbf{w}^\top \mathbf{x} + b)$	a line, squashed into a probability
Loss	cross-entropy	
Optimiser	gradient descent, unchanged	

A First Artificial Neuron

Logistic regression is a single neuron. Stack many, and you have a neural network.



stack many of these: the kind of model behind the snout's question mark

Three Lines of scikit-learn

The whole recipe, in three lines

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()      ← the model (its loss comes with it)

model.fit(X_train, y_train)       ← the optimiser, on the data

model.predict(X_new)              ← use it on new data
```

and model.score(X, y) says how well it does: tomorrow's topic

Hands-On 1

Gradient descent by hand, then the same in scikit-learn

Core	Exercises 1–4: the loss, its gradient, the loop (25 min)
Stretch	Exercises 5–7: three learning rates, scikit-learn, mini-batches
Challenge	Exercises 8–9: logistic regression from scratch

`github.com/myscience/intro-ml-mdmc-2026`

notebooks › N1_gradient_descent.ipynb › Open in Colab

Today in One Slide

Four things to take home

Which η blew up for you?

Today in One Slide

Four things to take home

Learning = data + model + loss + optimiser

The loss is one number for "how wrong"

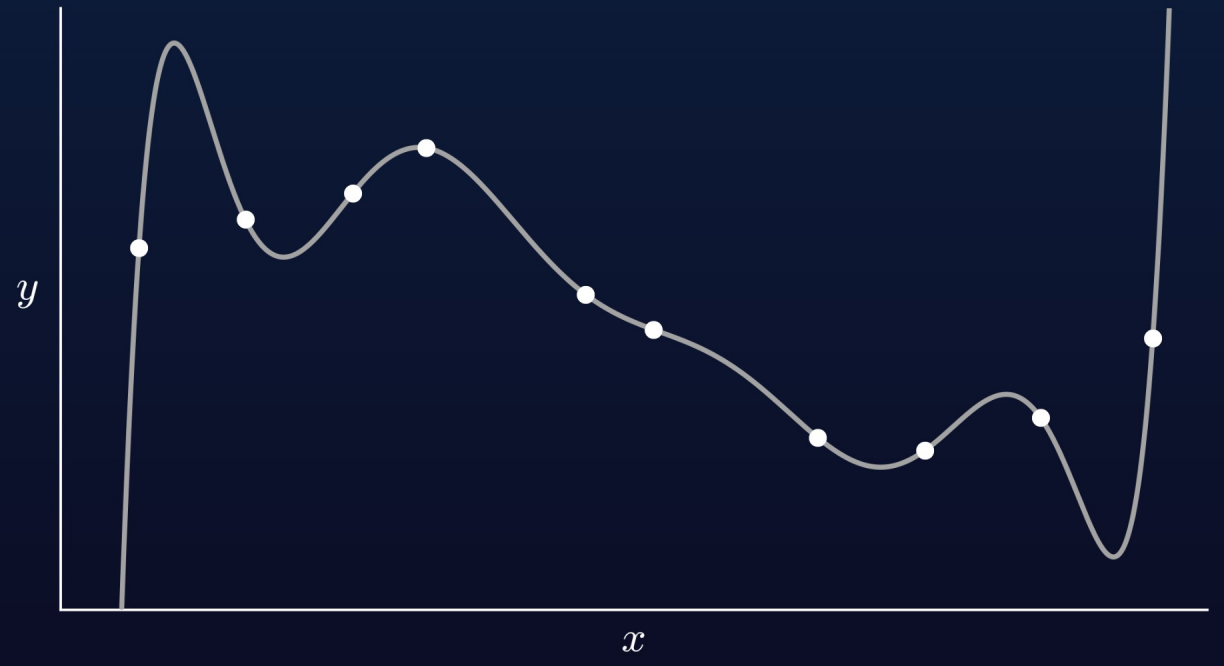
Gradient descent rolls downhill, one small step at a time

Linear and logistic regression: the recipe's simplest instances

curator's corner, today: wrong labels, one bad value, mismatched units

Tomorrow

We can fit a model. Will it work on data it has never seen?



$$L_{\text{train}} = 0$$

$$\hat{y} = w_1 x + w_2 x^2 + \cdots + w_9 x^9 + b$$ *nine features, this morning's recipe*

this curve passes through every point. Is it a good model?

AREA SCIENCE PARK · SISSA · OGS Istituto Nazionale di Oceanografia e di Geofisica Sperimentale · ARCTUR · Regione Autonoma Friuli Venezia Giulia · E-ARGO

Introduction to Machine Learning · Day 1

Thank You

Tomorrow: Not Fooling Yourself